UNIVERSIDAD DEL VALLE

750027C - Desarrollo de Software III

Guía académica

Dojo: API REST de Matrícula Académica

Aplicación de buenas prácticas para desarrollo de software en la construcción de un API

Stack: Python 3.12 · FastAPI · Django REST · PostgreSQL · pytest · ruff · mypy · Docker · GitHub Actions · EC2 · SonarCloud

Presentado por	**PhD. Manuel Alejandro Pastrana Pardo** Profesor Nombrado — EISC, Facultad de Ingeniería Email: manuel.pastrana@correounivalle.edu.co
Tema	Implementación de microservicios
Lugar y año	Cali, Colombia — 2026

Paso 1 — Entendiendo el dojo

1.1 ¿Qué construiremos?

Construirás dos microservicios Python independientes que exponen una API REST para gestionar la matrícula académica. El sistema permitirá registrar profesores, estudiantes y cursos, y gestionar qué profesor dicta cada curso y qué estudiantes están matriculados en él.

A diferencia del dojo Java que era un único monolito Spring Boot, aquí separamos las responsabilidades en dos servicios:

Microservicio	Framework	Responsabilidad	Puerto
profesor-service	FastAPI	Gestiona profesores: crear, listar, actualizar, eliminar.	8001
matricula-service	Django REST Framework	Gestiona estudiantes, cursos y matrículas.	8002

Al finalizar tendrás un proyecto profesional con:

- Dos APIs REST documentadas y navegables desde el navegador (Swagger/OpenAPI)
- Base de datos PostgreSQL gestionada con migraciones versionadas
- Tests unitarios y de integración con cobertura verificada (≥ 80 %)
- Calidad de código con ruff (linting y formato) y mypy (tipos estáticos)
- Pipeline de integración continua que valida calidad en cada Pull Request
- Despliegue automático en EC2 de AWS al aprobar un PR a main
- Análisis estático de código publicado en SonarCloud

1.2 Diferencias clave respecto al dojo Java

Si ya hiciste el dojo Java, estas son las diferencias más importantes que encontrarás:

Aspecto	Java / Spring Boot	Python (este dojo)
Swagger / OpenAPI	Necesita dependencia springdoc	FastAPI lo genera automáticamente. Django necesita drf-spectacular.
Migraciones BD	Flyway — archivos .sql numerados	Alembic (FastAPI) · Django migrations (manage.py)
Linting y formato	Checkstyle con reglas Google	ruff — reemplaza flake8 + black + isort en una sola herramienta
Tipos estáticos	Java ya es tipado fuertemente	mypy analiza los type hints de Python
Tests integración	Testcontainers (JUnit 5)	testcontainers-python + pytest
ORM	Spring Data JPA + Hibernate	SQLAlchemy (FastAPI) · Django ORM (matricula-service)

1.3 Modelo de datos

Los dos microservicios comparten la misma base de datos PostgreSQL. Las entidades son:

Entidad	Servicio dueño	Campos principales
Profesor	profesor-service (FastAPI)	id, nombre, apellido, email (único), identificacion (único)
Estudiante	matricula-service (Django)	id, nombre, apellido, email (único), identificacion (único), semestre (1–10)
Curso	matricula-service (Django)	id, codigo (único), nombre, creditos, profesor_id (FK a Profesor)
Matricula	matricula-service (Django)	Tabla intermedia curso_id + estudiante_id (relación muchos a muchos)

1.4 Prerrequisitos

Herramienta	Versión mínima	Cómo verificar
Python	3.12.x	python --version
pip	24.x	pip --version
Docker Desktop	24.x	docker --version
Docker Compose	2.x	docker compose version
Git	2.x	git --version
VS Code o PyCharm	Cualquier versión reciente	Abrir el IDE

Cuentas necesarias antes de comenzar:

Servicio	URL	Para qué se usa
GitHub	github.com	Repositorio, ramas y pipelines CI/CD
Docker Hub	hub.docker.com	Almacenar las imágenes Docker de cada servicio
SonarCloud	sonarcloud.io	Análisis estático de calidad y cobertura
AWS	aws.amazon.com	Servidor EC2 donde se despliega en producción

> *⚠ Docker Desktop debe estar activo y corriendo cada vez que ejecutes pytest con testcontainers. Porque los tests de integración levantan un contenedor PostgreSQL real en tu máquina.*

Paso 2 — Estructura del proyecto

2.1 Crear el repositorio en GitHub

Ve a github.com y crea un nuevo repositorio público llamado dojo-python-matricula. No inicialices con README. Luego, clónalo en la ruta de preferencia:

```
git clone https://github.com/TU_USUARIO/dojo-python-matricula.git
cd dojo-python-matricula
```

2.2 Estructura de carpetas

El proyecto es un monorepo con dos microservicios independientes. La estructura completa es la siguiente:

```
dojo-python-matricula/
├── profesor-service/          # Microservicio FastAPI
│   ├── src/
│   │   └── profesor/
│   │       ├── __init__.py
│   │       ├── main.py        # Punto de entrada de la app FastAPI
│   │       ├── config.py      # Variables de entorno con Pydantic Settings
│   │       ├── database.py    # Motor SQLAlchemy y sesión
│   │       ├── models.py      # Modelo ORM (tabla profesores)
│   │       ├── schemas.py     # Esquemas Pydantic (entrada/salida)
│   │       ├── crud.py        # Lógica de negocio y acceso a datos
│   │       ├── exceptions.py  # Excepciones propias del servicio
│   │       └── routers/
│   │           ├── __init__.py
│   │           └── profesores.py  # Endpoints REST
│   ├── tests/
│   │   ├── __init__.py
│   │   ├── conftest.py        # Fixtures compartidos
│   │   ├── unit/              # Tests unitarios (SQLite en memoria)
│   │   └── integration/       # Tests de integración (PostgreSQL real)
│   ├── alembic/               # Migraciones de base de datos
│   ├── Dockerfile
│   └── pyproject.toml
├── matricula-service/         # Microservicio Django REST Framework
│   ├── src/
│   │   ├── manage.py
│   │   ├── config/            # Configuración del proyecto Django
│   │   │   ├── __init__.py
│   │   │   ├── settings.py
│   │   │   └── urls.py
│   │   ├── estudiantes/       # App Django: estudiantes
│   │   └── cursos/            # App Django: cursos y matrículas
│   ├── tests/
│   ├── Dockerfile
│   └── pyproject.toml
├── docker-compose.yml         # Entorno de desarrollo local
├── docker-compose.prod.yml    # Entorno de producción en EC2
└── .github/
    └── workflows/
        ├── ci.yml             # Pipeline de integración continua
        └── cd.yml             # Pipeline de despliegue continuo
```

2.3 Crear la estructura de carpetas

Ejecuta estos comandos desde la raíz del repositorio para crear todas las carpetas y archivos __init__.py necesarios:

```
# Estructura del profesor-service
mkdir -p profesor-service/src/profesor/routers
mkdir -p profesor-service/tests/unit
mkdir -p profesor-service/tests/integration
mkdir -p profesor-service/alembic/versions

# Estructura del matricula-service
mkdir -p matricula-service/src/config
mkdir -p matricula-service/tests
mkdir -p .github/workflows

# Archivos __init__.py para que Python reconozca los paquetes
touch profesor-service/src/profesor/__init__.py
touch profesor-service/src/profesor/routers/__init__.py
touch profesor-service/tests/__init__.py
touch profesor-service/tests/unit/__init__.py
touch profesor-service/tests/integration/__init__.py
```

> *⚠ En Windows con Git Bash usa el mismo comando touch. Si da error, crea los archivos __init__.py manualmente desde VS Code: clic derecho sobre la carpeta → New File → __init__.py*

Paso 3 — profesor-service con FastAPI

El profesor-service es un microservicio independiente construido con FastAPI. FastAPI tiene varias ventajas sobre Spring Boot para APIs simples: genera Swagger automáticamente, es muy rápido de escribir y los type hints de Python actúan como documentación viva.

3.1 Dependencias — pyproject.toml

El archivo pyproject.toml es el equivalente Python del build.gradle de Java. Es quien define las dependencias, las reglas de calidad (ruff, mypy) y la configuración de pytest.

Iniciemos entonces por crea el archivo pyproject.toml dentro de la carpeta de profesor-service con el siguiente contenido. Recuerda que cada sección tiene un propósito específico que lo explicaremos en los comentarios:

profesor-service/pyproject.toml

```
[project]
name = "profesor-service"
version = "1.0.0"
requires-python = ">=3.12"
dependencies = [
    "fastapi==0.115.5",
    "uvicorn[standard]==0.32.1",     # Servidor ASGI para correr FastAPI
    "sqlalchemy==2.0.36",            # ORM equivalente a JPA/Hibernate
    "alembic==1.14.0",               # Migraciones de BD (equivalente a Flyway)
    "psycopg2-binary==2.9.10",       # Driver PostgreSQL
    "pydantic[email]==2.10.3",       # Validación de datos + soporte EmailStr
(requiere email-validator)
    "pydantic-settings==2.6.1",      # Leer variables de entorno (.env)
]
```

```
[project.optional-dependencies]
dev = [
    "pytest==8.3.4",
    "pytest-cov==6.0.0",            # Cobertura de código
    "httpx==0.28.1",                # Cliente HTTP para tests (TestClient de
FastAPI lo requiere)
    "testcontainers[postgres]==4.9.0", # PostgreSQL real en tests de integración
    "ruff==0.8.4",                  # Linting + formato (reemplaza flake8 + black +
isort)
    "mypy==1.13.0",                 # Verificación de tipos estáticos
    "types-psycopg2==2.9.21.20241019", # Stubs de tipos para psycopg2
]

[tool.ruff]
line-length = 100
target-version = "py312"

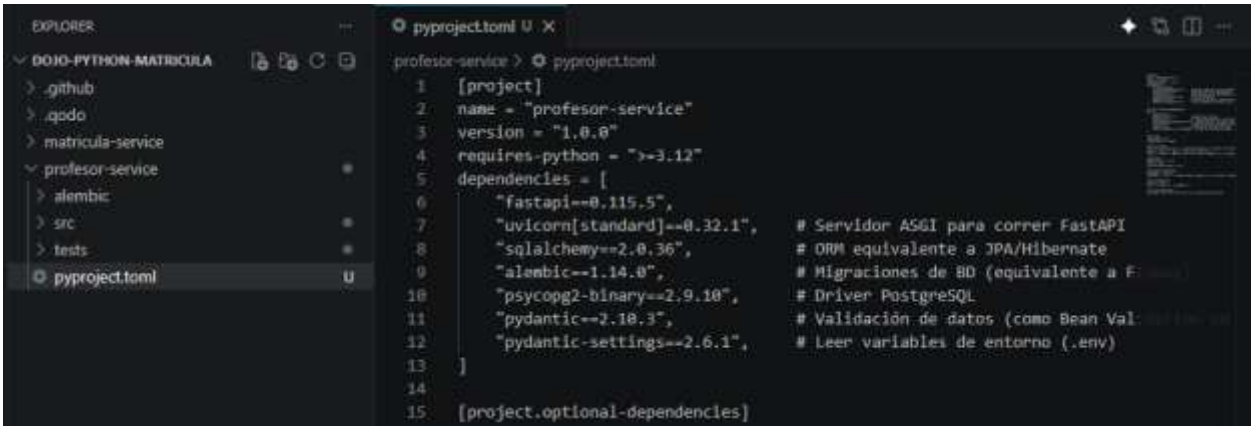
[tool.ruff.lint]
# E,F,W: errores básicos | I: imports ordenados | N: nombres | UP: modernizar | B:
bugs | C4: comprehensions
select = ["E", "F", "W", "I", "N", "UP", "B", "C4"]
ignore = ["B008"]  # B008: no aplica en FastAPI (Depends() es un pattern válido)

[tool.mypy]
python_version = "3.12"
strict = true
ignore_missing_imports = true

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]  # Permite importar desde src/ sin instalar el paquete

[tool.coverage.run]
source = ["src"]
omit = ["*/tests/*", "*/alembic/*"]

[tool.coverage.report]
fail_under = 80  # El build falla si la cobertura baja del 80%
```



3.2 Instalar las dependencias

Como se requiere que cada servicio mantenga sus dependencias de forma independiente, es necesario crear un entorno virtual para la ejecución del proyecto. Para hacer esto, desde la carpeta profesor-service debemos abrir la terminal para ejecutar los siguientes comandos:

```
cd profesor-service

# Crear entorno virtual (equivalente al caché de Gradle)
python -m venv .venv

# Activar el entorno virtual
# En Windows:
.venv\Scripts\activate
# En Mac/Linux:
source .venv/Scripts/activate

# Instalar todas las dependencias incluyendo las de desarrollo en el entorno
python -m pip install -e ".[dev]"

# Verificar que FastAPI y ruff quedaron instalados
python -c "import fastapi; print(fastapi.__version__)"
ruff --version
```

3.3 Variables de entorno — src/profesor/config.py

Este archivo lee las variables de entorno (o un archivo .env) usando Pydantic Settings, sería el equivalente al application.yml de Spring Boot pero más explícito porque cada cada variable queda documentada con su tipo. Para crearlo, en VS Code, dentro del panel Explorer, se abre la carpeta profesor, que se encuentra dentro de src, en profesor-service (profesor-service → src → profesor). Luego, dar clic derecho sobre la carpeta profesor, opción new File y escribir config.py. Finalizar con un enter. Creado el archivo pegar el siguiente contenido:

profesor-service/src/profesor/config.py

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    """Configuración de la aplicación leída desde variables de entorno o .env."""

    # URL de conexión a PostgreSQL. Formato:
postgresql://usuario:password@host:puerto/bd
    database_url: str =
"postgresql://profesor_user:profesor_pass@localhost:5432/profesor_db"
    app_name: str = "profesor-service"
    debug: bool = False

    # Lee primero las variables de entorno del sistema, luego el archivo .env
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")


# Instancia global — se importa desde otros módulos
settings = Settings()
```

3.4 Conexión a la base de datos — src/profesor/database.py

SQLAlchemy es el ORM que usamos, equivalente a JPA + Hibernate en Java. Este archivo configura el motor de conexión y la sesión de base de datos. Dentro de los elementos que se encuentran en el archivo de configuración, resaltaremos que la función get_db() es una

dependencia FastAPI que se inyecta automáticamente en los endpoints que la necesiten, igual que @Autowired en Spring Boot.

Para crear el archivo es necesario abrir profesor-service, dentro abrir la carpeta src y sobre la carpeta de profesor dar clic derecho, new file y colocar como nombre database.py. Finalizar con un enter y pegar el siguiente contenido en el archivo recién creado.

profesor-service/src/profesor/database.py

```
from collections.abc import Generator

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

from profesor.config import settings

# Equivalente al DataSource de Spring Boot
engine = create_engine(settings.database_url, pool_pre_ping=True)

# Fábrica de sesiones — equivalente a la inyección del EntityManager
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)


class Base(DeclarativeBase):
    """Clase base de la que heredan todos los modelos ORM."""


def get_db() -> Generator[Session, None, None]:
    """
    Dependencia FastAPI: abre una sesión de BD, la entrega al endpoint
    y la cierra al terminar — incluso si hay un error.
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

3.5 Modelo ORM — src/profesor/models.py

El modelo ORM define la estructura de la tabla que está en la base de datos en memoria. En Java usábamos @Entity con anotaciones JPA; aquí usamos SQLAlchemy con Python. Creemos entonces, nuestro primer modelo.

profesor-service/src/profesor/models.py

```
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from profesor.database import Base


class Profesor(Base):
    """
    Modelo ORM que mapea a la tabla 'profesores' en PostgreSQL.
    Equivalente a @Entity @Table(name='profesores') en Java.
    """

    __tablename__ = "profesores"

    # Clave primaria autoincremental
```

```
    id: Mapped[int] = mapped_column(primary_key=True, index=True)

    # String(100) equivale a VARCHAR(100) en la BD
    nombre: Mapped[str] = mapped_column(String(100), nullable=False)
    apellido: Mapped[str] = mapped_column(String(100), nullable=False)

    # unique=True garantiza que no haya dos profesores con el mismo email
    email: Mapped[str] = mapped_column(String(200), unique=True, nullable=False,
index=True)
    identificacion: Mapped[str] = mapped_column(String(50), unique=True,
nullable=False)
```

3.6 Esquemas Pydantic — src/profesor/schemas.py

Los schemas Pydantic son el equivalente a los DTOs (Data Transfer Objects) de Java. Definen qué datos acepta cada endpoint (entrada) y qué datos retorna (salida), con validación automática.

profesor-service/src/profesor/schemas.py

```
from pydantic import BaseModel, EmailStr, Field


class ProfesorBase(BaseModel):
    """Campos comunes que comparten los schemas de creación y respuesta."""

    nombre: str = Field(..., min_length=2, max_length=100)
    apellido: str = Field(..., min_length=2, max_length=100)
    email: EmailStr  # Valida automáticamente que sea un email válido
    identificacion: str = Field(..., min_length=5, max_length=50)


class ProfesorCreate(ProfesorBase):
    """Schema de entrada para crear un profesor (POST /api/v1/profesores)."""

    # Hereda todos los campos de ProfesorBase sin cambios


class ProfesorUpdate(BaseModel):
    """Schema de entrada para actualizar un profesor (PUT /api/v1/profesores/{id}).
    Todos los campos son opcionales: solo se actualizan los que se envíen.
    """

    nombre: str | None = Field(None, min_length=2, max_length=100)
    apellido: str | None = Field(None, min_length=2, max_length=100)
    email: EmailStr | None = None
    identificacion: str | None = Field(None, min_length=5, max_length=50)


class ProfesorResponse(ProfesorBase):
    """Schema de salida: incluye el id asignado por la BD.
    Equivalente al @ResponseBody con DTO de salida en Spring Boot.
    """

    id: int

    # from_attributes=True permite convertir un objeto ORM (Profesor) a este schema
    model_config = {"from_attributes": True}
```

3.7 Excepciones propias — src/profesor/exceptions.py

Definimos dos excepciones propias para distinguir los tipos de error en la capa de negocio. El router las captura y las traduce al código HTTP correspondiente.

profesor-service/src/profesor/exceptions.py

```
class BusinessError(Exception):
    """
    Se lanza cuando se viola una regla de negocio.
    Ejemplo: intentar crear un profesor con email ya registrado.
    El router la convierte en HTTP 409 Conflict.
    """
class ResourceNotFoundError(Exception):
    """
    Se lanza cuando no se encuentra el recurso solicitado.
    Ejemplo: buscar un profesor con id que no existe.
    El router la convierte en HTTP 404 Not Found.
    """
```

3.8 Lógica de negocio — src/profesor/crud.py

El módulo crud.py contiene toda la lógica de negocio y el acceso a datos. Es el equivalente a la capa @Service de Spring Boot. La palabra CRUD viene de Create, Read, Update, Delete.

profesor-service/src/profesor/crud.py

```
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from profesor import models, schemas
from profesor.exceptions import BusinessError, ResourceNotFoundError

def crear_profesor(db: Session, data: schemas.ProfesorCreate) -> models.Profesor:
    """Crea un profesor validando que email e identificación sean únicos."""
    # Verificar unicidad ANTES de insertar para dar un mensaje de error claro
    if db.query(models.Profesor).filter_by(email=data.email).first():
        raise BusinessError(f"Ya existe un profesor con email: {data.email}")
    if
db.query(models.Profesor).filter_by(identificacion=data.identificacion).first():
        raise BusinessError(f"Ya existe un profesor con identificación:
{data.identificacion}")
    profesor = models.Profesor(**data.model_dump())
    db.add(profesor)
    try:
        db.commit()
        db.refresh(profesor)  # Recarga el objeto con el id asignado por la BD
    except IntegrityError as e:
        db.rollback()
        raise BusinessError("Error de integridad en la base de datos") from e
    return profesor

def obtener_profesores(db: Session, skip: int = 0, limit: int = 100) ->
list[models.Profesor]:
    """Retorna todos los profesores con paginación."""
    return db.query(models.Profesor).offset(skip).limit(limit).all()

def obtener_profesor(db: Session, profesor_id: int) -> models.Profesor:
    """Busca un profesor por id. Lanza ResourceNotFoundException si no existe."""
    profesor = db.query(models.Profesor).filter_by(id=profesor_id).first()
    if not profesor:
        raise ResourceNotFoundError(f"Profesor con id {profesor_id} no encontrado")
    return profesor
```

```
def actualizar_profesor(
    db: Session, profesor_id: int, data: schemas.ProfesorUpdate
) -> models.Profesor:
    """Actualiza solo los campos enviados en el body (PATCH semántico)."""
    profesor = obtener_profesor(db, profesor_id)
    # exclude_unset=True: solo procesa los campos que el cliente realmente envió
    update_data = data.model_dump(exclude_unset=True)
    for key, value in update_data.items():
        setattr(profesor, key, value)
    db.commit()
    db.refresh(profesor)
    return profesor

def eliminar_profesor(db: Session, profesor_id: int) -> None:
    """Elimina un profesor. Lanza ResourceNotFoundException si no existe."""
    profesor = obtener_profesor(db, profesor_id)
    db.delete(profesor)
    db.commit()
```

3.9 Endpoints REST — src/profesor/routers/profesores.py

El router define los endpoints HTTP. Es el equivalente al @RestController de Spring Boot. FastAPI utiliza decoradores (@router.get, @router.post, etc.) para mapear los métodos HTTP a funciones Python.

profesor-service/src/profesor/routers/profesores.py

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from profesor import crud, schemas
from profesor.database import get_db
from profesor.exceptions import BusinessError, ResourceNotFoundError

# prefix: todas las rutas de este router empiezan con /api/v1/profesores
# tags: agrupa los endpoints en la documentación Swagger
router = APIRouter(prefix="/api/v1/profesores", tags=["Profesores"])

# HTTP POST → Crear un nuevo profesor
@router.post(
    "",
    response_model=schemas.ProfesorResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Crear un profesor",
)
def crear(
    data: schemas.ProfesorCreate,  # FastAPI valida automáticamente el body
    db: Session = Depends(get_db),  # Inyección de dependencia de la sesión BD
) -> schemas.ProfesorResponse:
    """Crea un nuevo profesor. Retorna 409 si email o identificación ya existen."""
    try:
        return schemas.ProfesorResponse.model_validate(crud.crear_profesor(db,
data))
    except BusinessError as exc:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail=str(exc))
from exc

# HTTP GET → Listar todos los profesores
@router.get("", response_model=list[schemas.ProfesorResponse], summary="Listar
profesores")
```

```
def listar(
    skip: int = 0,
    limit: int = 100,
    db: Session = Depends(get_db),
) -> list[schemas.ProfesorResponse]:
    """Retorna la lista de todos los profesores registrados."""
    return [
        schemas.ProfesorResponse.model_validate(p) for p in
crud.obtener_profesores(db, skip, limit)
    ]

# HTTP GET → Obtener un profesor por id
@router.get("/{profesor_id}", response_model=schemas.ProfesorResponse,
summary="Obtener profesor")
def obtener(
    profesor_id: int,
    db: Session = Depends(get_db),
) -> schemas.ProfesorResponse:
    """Retorna un profesor por su id. Retorna 404 si no existe."""
    try:
        return schemas.ProfesorResponse.model_validate(crud.obtener_profesor(db,
profesor_id))
    except ResourceNotFoundError as exc:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=str(exc))
from exc

# HTTP PUT → Actualizar un profesor
@router.put(
    "/{profesor_id}", response_model=schemas.ProfesorResponse, summary="Actualizar
profesor"
)
def actualizar(
    profesor_id: int,
    data: schemas.ProfesorUpdate,
    db: Session = Depends(get_db),
) -> schemas.ProfesorResponse:
    """Actualiza los campos enviados. Los campos no enviados no se modifican."""
    try:
        return schemas.ProfesorResponse.model_validate(
            crud.actualizar_profesor(db, profesor_id, data)
        )
    except ResourceNotFoundError as exc:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=str(exc))
from exc

# HTTP DELETE → Eliminar un profesor
@router.delete(
    "/{profesor_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    summary="Eliminar profesor",
)
def eliminar(
    profesor_id: int,
    db: Session = Depends(get_db),
) -> None:
    """Elimina un profesor. Retorna 204 No Content si tuvo éxito, 404 si no
existe."""
    try:
        crud.eliminar_profesor(db, profesor_id)
    except ResourceNotFoundError as exc:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=str(exc))
from exc
```

3.10 Aplicación principal — src/profesor/main.py

Este es el punto de entrada de la aplicación FastAPI. Equivale a la clase DemoDsUnivalleApplication.java con @SpringBootApplication.

profesor-service/src/profesor/main.py

```
from fastapi import FastAPI

from profesor.database import Base, engine
from profesor.routers import profesores

# Crear la instancia de la aplicación FastAPI
# docs_url: ruta donde estará disponible el Swagger UI
app = FastAPI(
    title="Profesor Service",
    description="Microservicio de gestión de profesores — Matrícula Académica",
    version="1.0.0",
    docs_url="/profesor-service/docs",
    redoc_url="/profesor-service/redoc",
    openapi_url="/profesor-service/openapi.json",
)

# Crear las tablas en la BD al iniciar la app
# En producción, Alembic gestiona las migraciones en lugar de este comando
Base.metadata.create_all(bind=engine)

# Registrar el router de profesores
app.include_router(profesores.router)


@app.get("/profesor-service/health")
def health() -> dict[str, str]:
    """Endpoint de salud — usado por el pipeline CD para verificar el
despliegue."""
    return {"status": "UP", "service": "profesor-service"}
```

Al final, el VS code mostrará una estructura de archivos así:



3.11 Probar que el servicio levanta

Instala las dependencias si aún no lo has hecho y ejecuta el servidor de desarrollo:

```
cd profesor-service
python -m pip install -e ".[dev]"

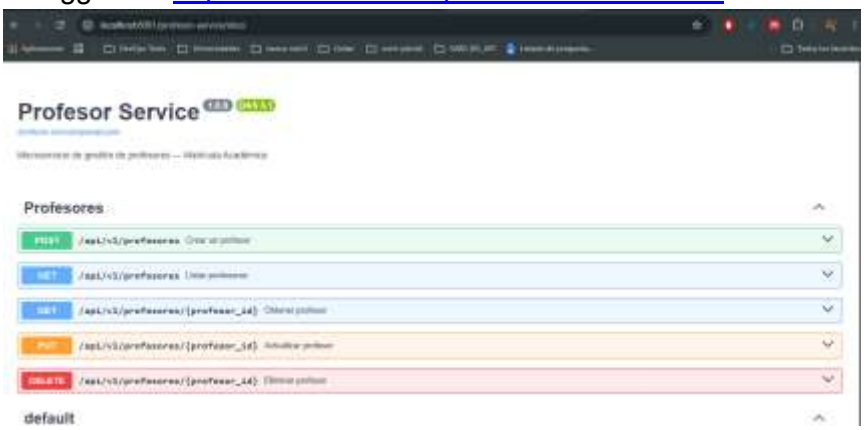
# Iniciar el servidor (equivalente a ./gradlew bootRun)
uvicorn profesor.main:app --reload --port 8001

# Resultado esperado:
# INFO:     Started server process
# INFO:     Waiting for application startup.
# INFO:     Application startup complete.
# INFO:     Uvicorn running on http://127.0.0.1:8001
```

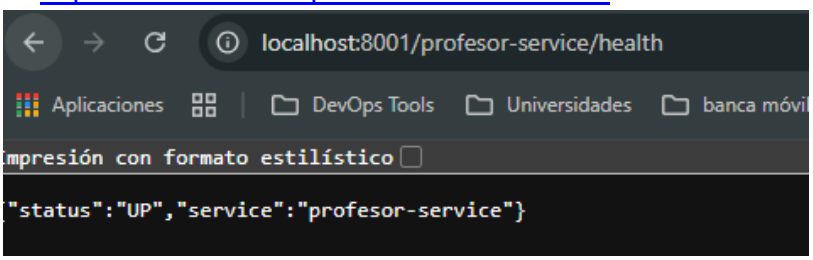
```
Manuel Pastrana@DESKTOP-7FO66S7 MINGW64 /f/Workspace/dojo-python-matricula/profesor-service (main)
$ uvicorn profesor.main:app --reload --port 8001
INFO:     Will watch for changes in these directories: ['F:\\Workspace\\dojo-python-matricula\\profesor-service']
INFO:     Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO:     Started reloader process [14600] using WatchFiles
INFO:     Started server process [10312]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Verifica que funciona abriendo en el navegador:

- Swagger UI: http://localhost:8001/profesor-service/docs



- Health check: http://localhost:8001/profesor-service/health



> ⚠ *Si ves un error de conexión a PostgreSQL al iniciar, es normal si no tienes PostgreSQL corriendo localmente. En el Paso 6 configuraremos Docker Compose para levantar la BD automáticamente. Por el momento y para probar pueden levantar un contenedor para su BD:*
>
> *docker run -d --name postgres-dev *
>
> *-e POSTGRES_DB=profesor_db *

```
-e POSTGRES_USER=profesor_user \
-e POSTGRES_PASSWORD=profesor_pass \
-p 5432:5432 \
postgres:16-alpine
```



Paso 4 — Tests del profesor-service

El proyecto tiene dos niveles de tests, igual que en el dojo Java:

Tipo de test	Base de datos	Velocidad	Qué verifica
Unitarios	SQLite en memoria (sin Docker)	Muy rápido (< 5 s)	Lógica de negocio del crud.py aislada
Integración	PostgreSQL real via Testcontainers (requiere Docker)	Más lento (30-60 s)	Endpoints HTTP completos contra BD real

4.1 Fixtures compartidos — tests/conftest.py

Un fixture es una función especial de pytest que prepara algo que un test necesita para validar el comportamiento de una función, como por ejemplo, una conexión a base de datos, un objeto de prueba, o un cliente HTTP. Esto se lo entrega al test como si fuera un parámetro de su función y así logra hacer el respectivo llamado.

La diferencia clave con @BeforeEach de JUnit es esta: en JUnit, @BeforeEach se ejecuta automáticamente antes de cada test de la clase, sin que el test "pida" nada explícitamente. En pytest, un test recibe un fixture declarándolo como parámetro de su función. Si el test no lo necesita, simplemente no lo incluye en su firma y pytest no lo ejecuta.

Creemos entonces, nuestro primer test.

profesor-service/tests/conftest.py

```
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool

from profesor.database import Base, get_db
from profesor.main import app

# Motor SQLite en memoria para tests unitarios — no requiere Docker
ENGINE = create_engine(
    "sqlite:///:memory:",
```

```
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,  # Una sola conexión compartida — necesario para SQLite
en memoria
)
TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=ENGINE)


@pytest.fixture(scope="function")
def db_session():
    """
    Crea y destruye las tablas para cada test unitario.
    Garantiza que cada test parte de una base de datos limpia.
    """
    Base.metadata.create_all(bind=ENGINE)
    session = TestingSessionLocal()
    try:
        yield session
    finally:
        session.close()
        Base.metadata.drop_all(bind=ENGINE)


@pytest.fixture(scope="function")
def client(db_session):
    """
    Cliente HTTP de prueba con la BD en memoria inyectada.
    Equivalente al TestRestTemplate de Spring Boot.
    """
    def override_get_db():
        try:
            yield db_session
        finally:
            pass

    app.dependency_overrides[get_db] = override_get_db
    with TestClient(app) as c:
        yield c
    app.dependency_overrides.clear()
```

4.2 Tests unitarios — tests/unit/test_crud.py

Un test unitario prueba **una sola pieza de código de forma aislada**. En este caso, vamos a probar las funciones del módulo crud.py (crear_profesor, obtener_profesor, etc.) llamándolas directamente desde Python, como funciones normales. Sin embargo, aquí nos saltamos el router y el protocolo HTTP por completo, en su lugar, llamamos crear_profesor(db_session, datos) directamente y verificamos lo que retorna o la excepción que lanza.

Esto es diferente de un test de integración, que, por el contrario, si hace una petición HTTP real (POST /api/v1/profesores) y prueba toda la cadena: router → crud → base de datos → respuesta JSON.

En este punto nos puede surgir la pregunta de ¿Por qué SQLite en memoria en lugar de PostgreSQL?. SQLite en memoria (sqlite:///:memory:) crea una base de datos completa que vive solo en la RAM mientras corre el test, y desaparece al terminar. Las ventajas que aporta cuando hacemos pruebas unitarias se pueden resumir así:

- **Velocidad**: no hay conexión de red ni disco (los 12 tests corren en menos de 1 segundo).

- **No requiere Docker**: puedes correr pytest tests/unit aunque Docker Desktop esté apagado.
- **Aislamiento total**: cada test crea las tablas desde cero y las destruye al final, así que un test nunca puede afectar a otro

Dicho esto, entonces ¿por qué seguimos necesitando PostgreSQL real (Paso 4.3)?. Resulta que SQLite y PostgreSQL no se comportan exactamente igual en todos los casos. Por ejemplo, el manejo de tipos de datos, restricciones UNIQUE, o el comportamiento ante ciertas transacciones puede variar ligeramente. Por eso, debemos separar las pruebas unitarias, de las de integración que evalúan de forma más completa toda la transacción. En resumen:

Aspectos	Tests unitarios (SQLite memoria)	Tests de integración (PostgreSQL real)
Qué prueban	Lógica de negocio de crud.py aislada	El sistema completo: HTTP + lógica + BD real
Velocidad	Muy rápida (< 1 s)	Más lenta (30-60 s, levanta un contenedor)
Requiere Docker	No	Sí (Testcontainers)
Detecta	Errores de lógica (validaciones, excepciones)	Errores de integración con la BD real (constraints, tipos, SQL)

Es la misma filosofía que en el dojo Java: los *ServiceTest corrían rápido con mocks, mientras que los *IntegrationTest levantaban un PostgreSQL real con Testcontainers para validar el comportamiento end-to-end.

profesor-service/tests/unit/test_crud.py

```
import pytest
from sqlalchemy.orm import Session

from profesor.crud import (
    actualizar_profesor,
    crear_profesor,
    eliminar_profesor,
    obtener_profesor,
    obtener_profesores,
)
from profesor.exceptions import BusinessException, ResourceNotFoundException
from profesor.schemas import ProfesorCreate, ProfesorUpdate


def _datos_validos(**overrides: str) -> ProfesorCreate:
    """Helper: construye datos de prueba válidos con valores por defecto."""
    defaults = defaults = {
        "nombre": "Carlos",
        "apellido": "Ramirez",
        "email": "carlos@univalle.edu.co",
        "identificacion": "12345678",
    }
    defaults.update(overrides)
    return ProfesorCreate(**defaults)
```

```
class TestCrearProfesor:
    def test_crear_con_datos_validos_retorna_profesor(self, db_session: Session) ->
None:
        profesor = crear_profesor(db_session, _datos_validos())
        assert profesor.id is not None
        assert profesor.email == "carlos@univalle.edu.co"

    def test_crear_con_email_duplicado_lanza_business_exception(self, db_session:
Session) -> None:
        crear_profesor(db_session, _datos_validos())
        with pytest.raises(BusinessException):
            crear_profesor(db_session, _datos_validos(identificacion="99999999"))

    def test_crear_con_identificacion_duplicada_lanza_business_exception(self,
db_session: Session) -> None:
        crear_profesor(db_session, _datos_validos())
        with pytest.raises(BusinessException):
            crear_profesor(db_session,
_datos_validos(email="otro@univalle.edu.co"))


class TestObtenerProfesor:
    def test_obtener_por_id_existente_retorna_profesor(self, db_session: Session) -
> None:
        creado = crear_profesor(db_session, _datos_validos())
        encontrado = obtener_profesor(db_session, creado.id)
        assert encontrado.id == creado.id

    def test_obtener_id_inexistente_lanza_not_found(self, db_session: Session) ->
None:
        with pytest.raises(ResourceNotFoundException):
            obtener_profesor(db_session, 999)

    def test_obtener_todos_retorna_lista_completa(self, db_session: Session) ->
None:
        crear_profesor(db_session, _datos_validos())
        crear_profesor(db_session, _datos_validos(email="otro@univalle.edu.co",
identificacion="99999"))
        assert len(obtener_profesores(db_session)) == 2


class TestActualizarProfesor:
    def test_actualizar_nombre_retorna_datos_actualizados(self, db_session:
Session) -> None:
        creado = crear_profesor(db_session, _datos_validos())
        actualizado = actualizar_profesor(db_session, creado.id,
ProfesorUpdate(nombre="Carlos Nuevo"))
        assert actualizado.nombre == "Carlos Nuevo"

    def test_actualizar_id_inexistente_lanza_not_found(self, db_session: Session) -
> None:
        with pytest.raises(ResourceNotFoundException):
            actualizar_profesor(db_session, 999, ProfesorUpdate(nombre="XYZ"))


class TestEliminarProfesor:
    def test_eliminar_existente_lo_borra_de_la_bd(self, db_session: Session) ->
None:
        creado = crear_profesor(db_session, _datos_validos())
        eliminar_profesor(db_session, creado.id)
```

```
        with pytest.raises(ResourceNotFoundException):
            obtener_profesor(db_session, creado.id)

    def test_eliminar_id_inexistente_lanza_not_found(self, db_session: Session) ->
None:
        with pytest.raises(ResourceNotFoundException):
            eliminar_profesor(db_session, 999)
```

4.3 Tests de integración — tests/integration/test_profesores_api.py

Los tests de integración levantan un PostgreSQL real usando Testcontainers y prueban los endpoints HTTP completos. Requieren que Docker Desktop esté corriendo.

profesor-service/tests/integration/test_profesores_api.py

```
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine, text
from sqlalchemy.orm import sessionmaker
from testcontainers.postgres import PostgresContainer

from profesor.database import Base, get_db
from profesor.main import app


@pytest.fixture(scope="module")
def postgres_engine():
    """
    Levanta un contenedor PostgreSQL real una sola vez para todo el módulo.
    Crea las tablas al inicio y las elimina al finalizar todos los tests.
    """
    with PostgresContainer("postgres:16-alpine") as pg:
        engine = create_engine(pg.get_connection_url())
        Base.metadata.create_all(bind=engine)
        yield engine
        Base.metadata.drop_all(bind=engine)


@pytest.fixture(scope="module")
def postgres_client(postgres_engine):
    """Cliente HTTP de prueba conectado al PostgreSQL del contenedor."""
    session = sessionmaker(autocommit=False, autoflush=False, bind=postgres_engine)

    def override_get_db():
        db = session()
        try:
            yield db
        finally:
            db.close()

    app.dependency_overrides[get_db] = override_get_db
    with TestClient(app) as client:
        yield client
    app.dependency_overrides.clear()


@pytest.fixture(autouse=True)
def limpiar_bd(postgres_engine):
    """
    Se ejecuta automáticamente ANTES de cada test (autouse=True).
    Vacía la tabla profesores para que cada test parta de una BD limpia,
    igual que el @BeforeEach limpiarBaseDeDatos() del dojo Java.
```

```
    """
    with postgres_engine.begin() as conn:
        conn.execute(text("DELETE FROM profesores"))


# Datos de prueba reutilizables en todos los tests
DATA = {
    "nombre": "Ana",
    "apellido": "Lopez",
    "email": "ana.lopez@univalle.edu.co",
    "identificacion": "87654321",
}


class TestProfesorAPIIntegracion:
    def test_crear_profesor_retorna_201(self, postgres_client: TestClient) -> None:
        r = postgres_client.post("/api/v1/profesores", json=DATA)
        assert r.status_code == 201
        assert r.json()["email"] == DATA["email"]

    def test_crear_email_duplicado_retorna_409(self, postgres_client: TestClient) -
> None:
        postgres_client.post("/api/v1/profesores", json=DATA)
        r = postgres_client.post("/api/v1/profesores", json=DATA)
        assert r.status_code == 409

    def test_listar_profesores_retorna_200(self, postgres_client: TestClient) ->
None:
        r = postgres_client.get("/api/v1/profesores")
        assert r.status_code == 200
        assert isinstance(r.json(), list)

    def test_obtener_por_id_retorna_200(self, postgres_client: TestClient) -> None:
        creado = postgres_client.post("/api/v1/profesores", json=DATA).json()
        r = postgres_client.get(f"/api/v1/profesores/{creado['id']}")
        assert r.status_code == 200

    def test_obtener_id_inexistente_retorna_404(self, postgres_client: TestClient)
-> None:
        r = postgres_client.get("/api/v1/profesores/999999")
        assert r.status_code == 404

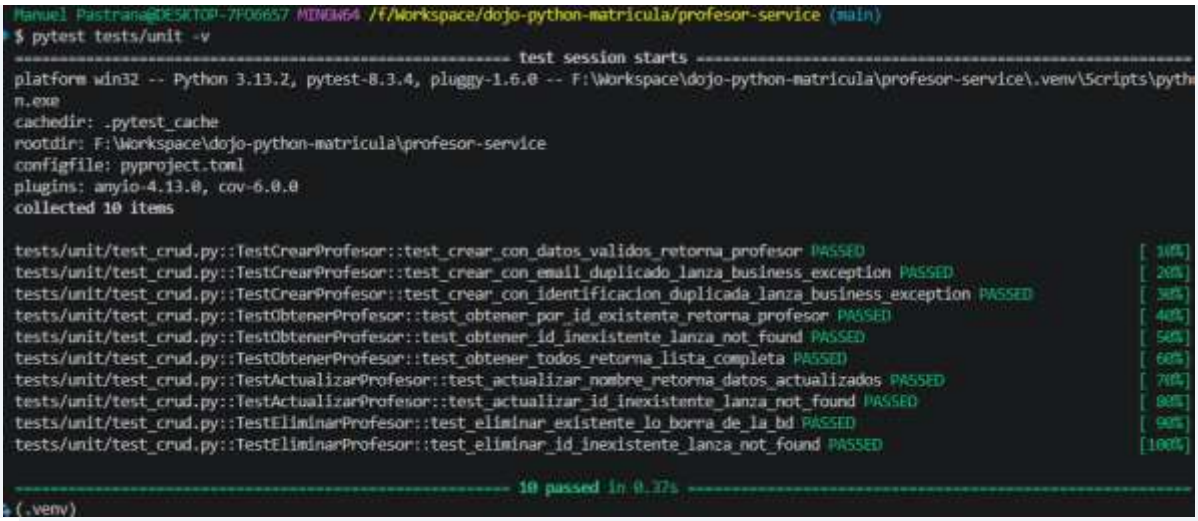
    def test_actualizar_retorna_200_con_datos_nuevos(self, postgres_client:
TestClient) -> None:
        creado = postgres_client.post("/api/v1/profesores", json=DATA).json()
        r = postgres_client.put(f"/api/v1/profesores/{creado['id']}",
json={"nombre": "Ana Nueva"})
        assert r.status_code == 200
        assert r.json()["nombre"] == "Ana Nueva"

    def test_eliminar_retorna_204(self, postgres_client: TestClient) -> None:
        creado = postgres_client.post("/api/v1/profesores", json=DATA).json()
        r = postgres_client.delete(f"/api/v1/profesores/{creado['id']}")
        assert r.status_code == 204
```

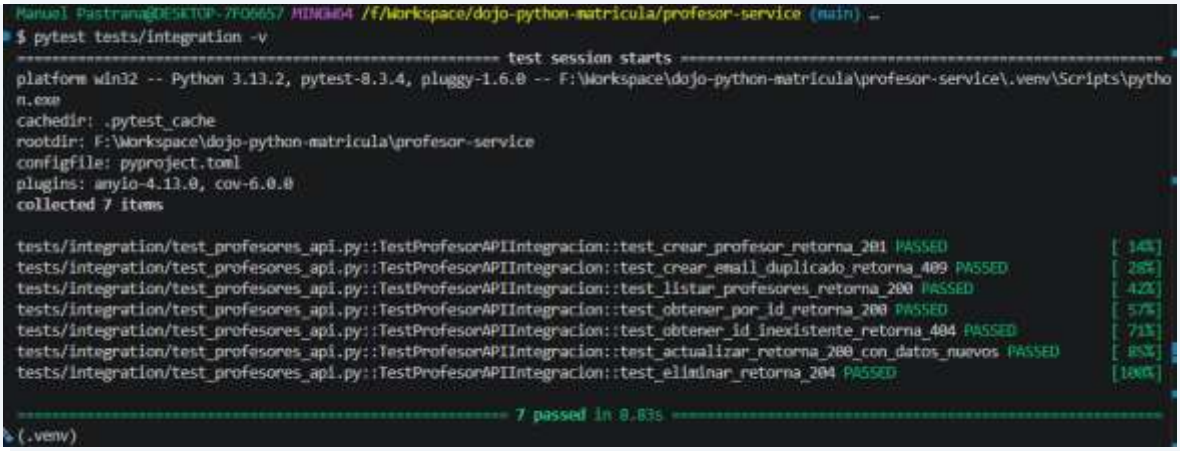
4.4 Ejecutar los tests y verificar cobertura

Desde la carpeta profesor-service con el entorno virtual activado:

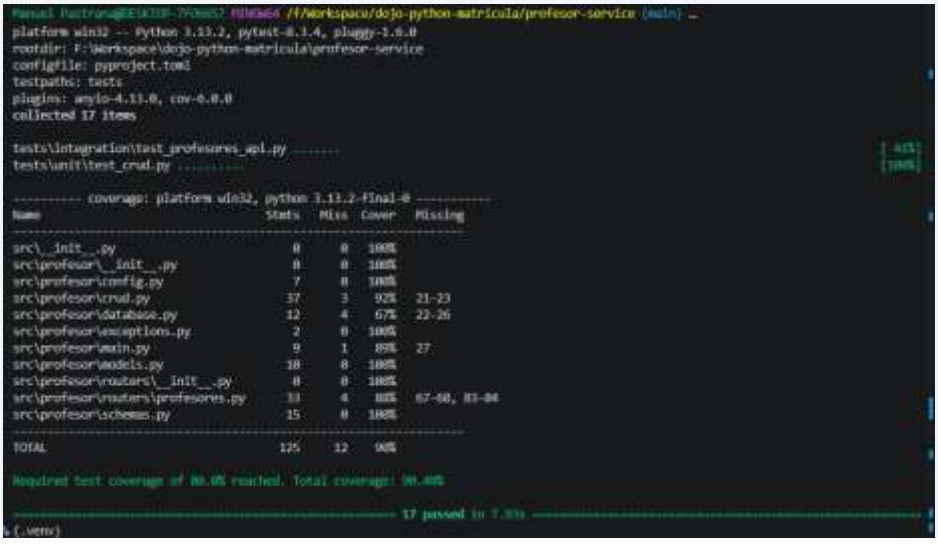
```
# Tests unitarios (rápidos, no requieren Docker)
pytest tests/unit -v
```



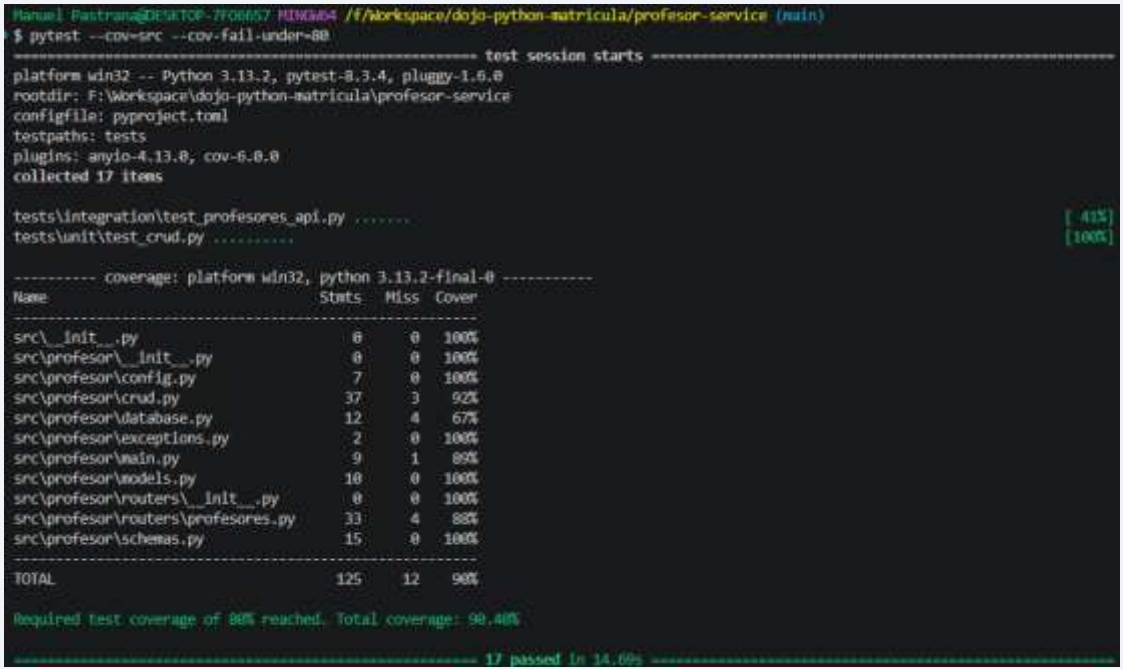

```
# Tests de integración (requiere Docker Desktop corriendo)
pytest tests/integration -v
```



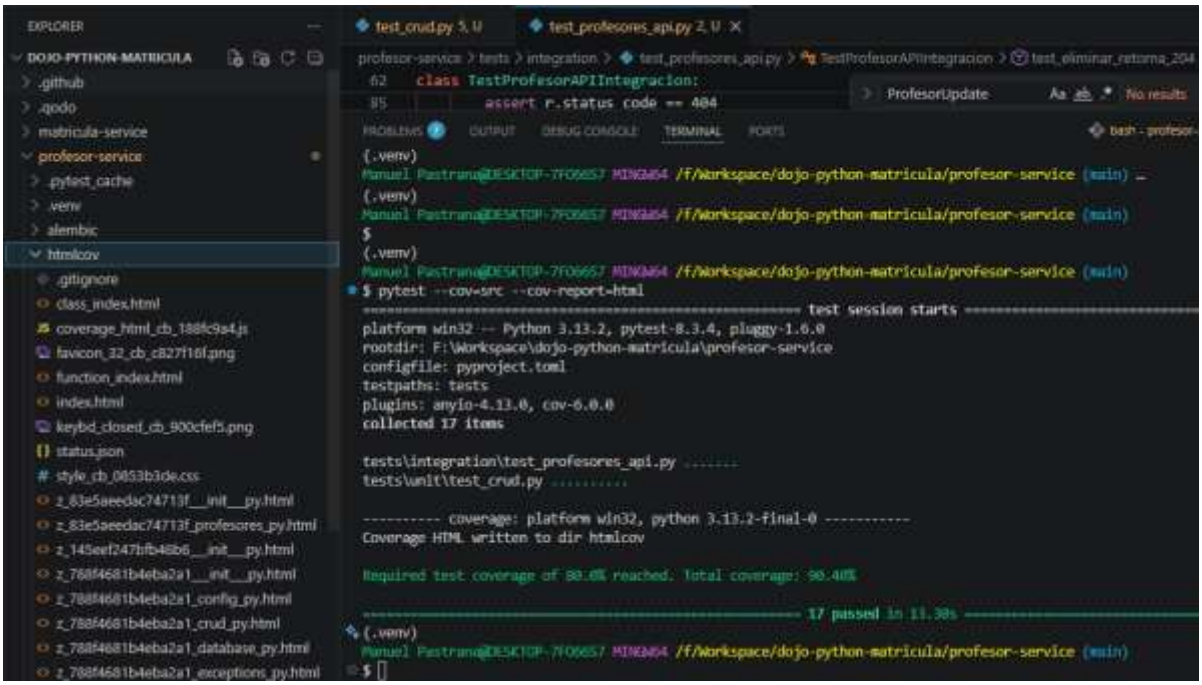

```
# Todos los tests con reporte de cobertura en terminal
pytest --cov=src --cov-report=term-missing
```



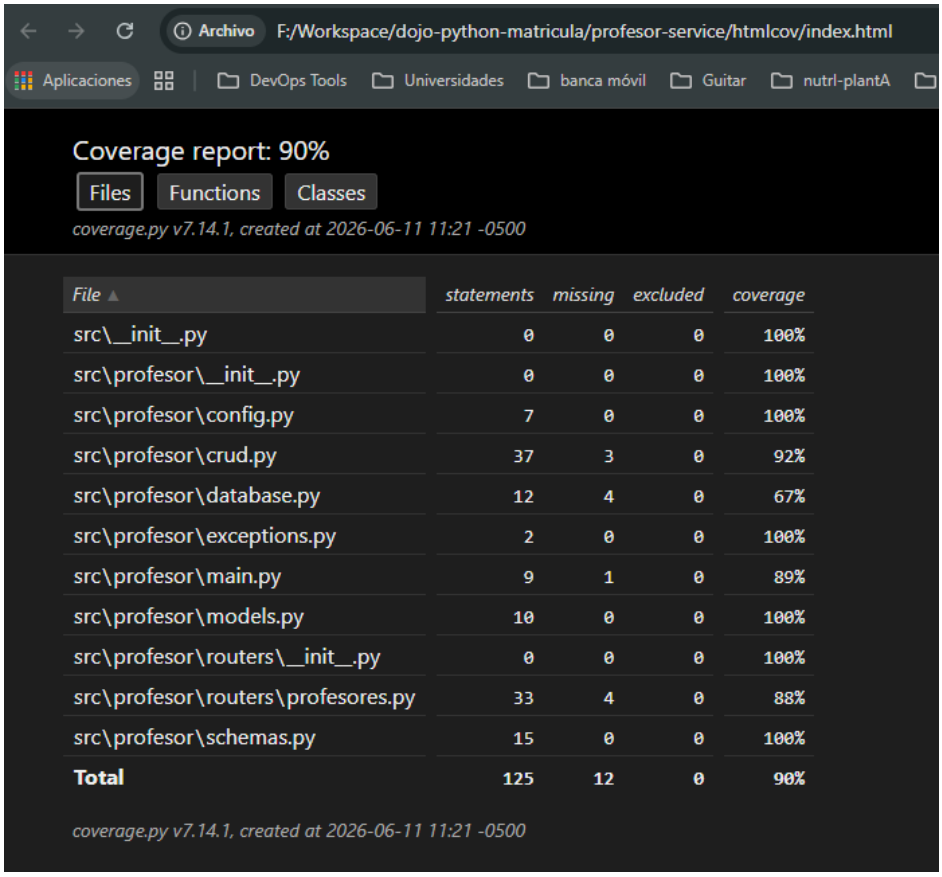
```
# Verificar que la cobertura es >= 80% (falla el build si no lo es)
pytest --cov=src --cov-fail-under=80
```




```
# Generar reporte HTML de cobertura
pytest --cov=src --cov-report=html
# El reporte queda en htmlcov/index.html
```



El reporte puede ser abierto desde el navegador. El contenido debe mostrar lo siguiente:



4.5 Ejecutar ruff y mypy

Una buena práctica para desarrollo de software es que siempre, antes de hacer commit, se debe ejecutar siempre las herramientas de calidad para verificar localmente que posibles mejoras pueden ser realizadas previo a la actualización del repositorio:

```
# Verificar y corregir el formato automáticamente
ruff format src tests
```

```
# Verificar errores de linting y corregir automáticamente
ruff check src tests --fix
# Resultado esperado: All checks passed.

# Verificar tipos estáticos
mypy src
# Resultado esperado: Success: no issues found in N source files

# Si mypy muestra errores falsos después de cambios, limpia la caché:
rm -rf .mypy_cache
mypy src
```

⚠ En Windows Git Bash: si pytest --cov reporta 0% de cobertura, ejecuta primero: export PYTHONPATH=src y luego vuelve a correr pytest.

Paso 5 — matricula-service con Django REST Framework

El matricula-service gestiona estudiantes, cursos y matrículas. Usamos Django REST Framework (DRF) porque Django incluye su propio ORM, sistema de migraciones y admin panel — ideal para entidades con relaciones complejas como la matrícula (muchos a muchos entre Curso y Estudiante).

5.1 Crear el proyecto Django

Desde la raíz del repositorio, entra a la carpeta matricula-service y crea el proyecto:

```
cd matricula-service

# Crear entorno virtual
python -m venv .venv

# Activar entorno virtual
.venv\Scripts\activate          # Windows
source .venv/Scripts/activate  # Mac/Linux

#Instalar las dependencias
python -m pip install -e ".[dev]"

# Instalar Django para poder usar django-admin
pip install django==5.1.4 djangorestframework==3.15.2

Es importante mencionar que este comando tiene dos partes que cumplen roles
distintos:

config → es el nombre del proyecto Django. Django usará este nombre para la carpeta
de configuración principal (settings.py, urls.py, wsgi.py). Lo llamamos config (en
lugar del nombre por defecto, que sería dojo_python_matricula o similar) porque es
una convención común: deja claro que esa carpeta contiene configuración del
proyecto, no lógica de negocio.

src → es la carpeta destino donde Django creará todo. Si no se especifica, Django
crea una subcarpeta extra con el nombre del proyecto y duplica la estructura. Al
pasar src, le decimos "crea los archivos directamente dentro de la carpeta src que
ya existe".

# Crear el proyecto Django dentro de la carpeta src/
django-admin startproject config src

# Entrar a src/ y crear las apps
cd src
python manage.py startapp estudiantes
python manage.py startapp cursos
```

Después de estos comandos la estructura de src/ queda así:

```
src/
├── manage.py
├── config/
│   ├── __init__.py
│   ├── settings.py    # Configuración principal de Django
│   ├── urls.py        # Rutas principales
│   └── wsgi.py
├── estudiantes/       # App creada con startapp
│   ├── models.py
│   ├── views.py
│   └── ...
└── cursos/            # App creada con startapp
    ├── models.py
    ├── views.py
    └── ...
```

5.2 Dependencias — pyproject.toml

Crea el archivo matricula-service/pyproject.toml:

matricula-service/pyproject.toml

```
[project]
name = "matricula-service"
version = "1.0.0"
requires-python = ">=3.12"
dependencies = [
    "django==5.1.4",
    "djangorestframework==3.15.2",
    "drf-spectacular==0.28.0",   # Genera Swagger/OpenAPI para Django (FastAPI lo
hace automático)
    "psycopg2-binary==2.9.10",   # Driver PostgreSQL
    "django-environ==0.11.2",    # Leer variables de entorno
]

[project.optional-dependencies]
dev = [
    "pytest==8.3.4",
    "pytest-django==4.9.0",      # Plugin pytest para Django
    "pytest-cov==6.0.0",
    "testcontainers[postgres]==4.9.0",
    "ruff==0.8.4",
    "mypy==1.13.0",
    "django-stubs[compatible-mypy]==5.1.1",         # Tipos de Django para mypy
    "djangorestframework-stubs==3.15.1",  # Tipos de DRF para mypy
]

[tool.ruff]
line-length = 100
target-version = "py312"

[tool.ruff.lint]
select = ["E", "F", "W", "I", "N", "UP", "B", "C4"]
ignore = ["B008", "N805"]  # N805: no aplica a métodos de ViewSet

[tool.mypy]
python_version = "3.12"
strict = true
ignore_missing_imports = true
plugins = ["mypy_django_plugin.main"]
```

```
[tool.django-stubs]
django_settings_module = "config.settings"

[tool.pytest.ini_options]
DJANGO_SETTINGS_MODULE = "config.settings"
testpaths = ["tests"]
pythonpath = ["src"]

[tool.coverage.run]
source = ["src"]
omit = ["*/migrations/*", "*/tests/*", "manage.py"]
```

5.3 Configuración — src/config/settings.py

Reemplaza el contenido del settings.py que se encuentra en la carpeta config, dentro de src y que fue generado por Django con este. Usamos django-environ para leer las variables de entorno de forma segura:

matricula-service/src/config/settings.py

```
import environ
from pathlib import Path

env = environ.Env(DEBUG=(bool, False))

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = env("SECRET_KEY", default="insecure-dev-key-cambiar-en-produccion")
DEBUG = env("DEBUG")
ALLOWED_HOSTS = env.list("ALLOWED_HOSTS", default=["*"])

INSTALLED_APPS = [
    "django.contrib.contenttypes",
    "django.contrib.auth",
    "django.contrib.staticfiles",  # Necesario para servir CSS/JS de Swagger UI
    "rest_framework",       # Django REST Framework
    "drf_spectacular",      # Generador de Swagger/OpenAPI
    "estudiantes",          # Nuestra app de estudiantes
    "cursos",               # Nuestra app de cursos
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.middleware.common.CommonMiddleware",
]

ROOT_URLCONF = "config.urls"

# Necesario para que drf-spectacular renderice la plantilla swagger_ui.html
# APP_DIRS=True hace que Django busque plantillas dentro de la carpeta
# 'templates/' de cada app instalada (incluyendo drf_spectacular).
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.debug",
                "django.template.context_processors.request",
            ],
        },
    },
```

```
]

# Leer la URL de conexión a PostgreSQL desde variable de entorno
DATABASES = {
    "default": env.db(
        "DATABASE_URL",

default="postgresql://matricula_user:matricula_pass@localhost:5432/matricula_db",
    )
}

DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"
LANGUAGE_CODE = "es-co"
TIME_ZONE = "America/Bogota"
USE_I18N = True
USE_TZ = True

# Ruta base para archivos estáticos (CSS/JS de Swagger UI, admin, etc.)
STATIC_URL = "/static/"

REST_FRAMEWORK = {
    "DEFAULT_RENDERER_CLASSES": ["rest_framework.renderers.JSONRenderer"],
    "DEFAULT_SCHEMA_CLASS": "drf_spectacular.openapi.AutoSchema",
}

SPECTACULAR_SETTINGS = {
    "TITLE": "Matricula Service",
    "DESCRIPTION": "Gestión de estudiantes, cursos y matrículas",
    "VERSION": "1.0.0",
}
```

5.4 Views para exponer el health del servicio — src/config/views.py

Reemplaza el contenido de urls.py. Aquí registramos el Swagger y las rutas de cada app:

matricula-service/src/config/views.py

```
from django.http import JsonResponse
from django.http import HttpRequest

def health(request: HttpRequest) -> JsonResponse:
    """Endpoint de salud — usado por el pipeline CD para verificar el
despliegue."""
    return JsonResponse({"status": "UP", "service": "matricula-service"})
```

5.5 Rutas principales — src/config/urls.py

Reemplaza el contenido de urls.py. Aquí registramos el Swagger y las rutas de cada app:

matricula-service/src/config/urls.py

```
from django.urls import include, path
from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView

from config.views import health

urlpatterns = [
    # Endpoint de salud
    path("matricula-service/health/", health, name="health"),
    # Swagger: schema JSON y UI visual
    path("matricula-service/schema/", SpectacularAPIView.as_view(), name="schema"),
    path(
        "matricula-service/docs/",
        SpectacularSwaggerView.as_view(url_name="schema"),
```

```
        name="swagger-ui",
    ),
    # Rutas de las apps
    path("api/v1/estudiantes/", include("estudiantes.urls")),
    path("api/v1/cursos/", include("cursos.urls")),
]
```

5.6 Modelo Estudiante — src/estudiantes/models.py

El modelo Django es más sencillo que SQLAlchemy: defines los campos directamente en la clase y Django genera las migraciones automáticamente con makemigrations.

matricula-service/src/estudiantes/models.py

```
from django.core.validators import MaxValueValidator, MinValueValidator
from django.db import models

class Estudiante(models.Model):
    """
    Modelo Django para la tabla estudiantes.
    No necesitas definir @Entity ni @Column — Django lo gestiona.
    """
    nombre = models.CharField(max_length=100)
    apellido = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    identificacion = models.CharField(max_length=50, unique=True)
    semestre = models.IntegerField(
        validators=[MinValueValidator(1), MaxValueValidator(10)]
    )

    class Meta:
        db_table = "estudiantes"
        ordering = ["apellido", "nombre"]

    def __str__(self) -> str:
        return f"{self.nombre} {self.apellido} ({self.email})"
```

5.7 Serializer — src/estudiantes/serializers.py

El serializer de Django REST es el equivalente al schema Pydantic de FastAPI. Convierte objetos Django a JSON y viceversa, con validación incluida.

matricula-service/src/estudiantes/serializers.py

```
from rest_framework import serializers
from .models import Estudiante

class EstudianteSerializer(serializers.ModelSerializer[Estudiante]):
    """
    Serializer completo para crear, actualizar y leer estudiantes.
    ModelSerializer infiere automáticamente los campos del modelo Django.
    """
    class Meta:
        model = Estudiante
        fields = "__all__"  # Incluye todos los campos del modelo

    def validate_semestre(self, value: int) -> int:
        """Validación adicional: el semestre debe estar entre 1 y 10."""
        if not 1 <= value <= 10:
            raise serializers.ValidationError("El semestre debe estar entre 1 y
10.")
        return value
```

5.8 ViewSet — src/estudiantes/views.py

El ViewSet de Django REST Framework es el equivalente al @RestController de Spring Boot. Con ModelViewSet obtienes automáticamente los endpoints GET, POST, PUT, PATCH y DELETE sin escribir código adicional.

matricula-service/src/estudiantes/views.py

```
from typing import Any

from rest_framework import status, viewsets
from rest_framework.request import Request
from rest_framework.response import Response

from .models import Estudiante
from .serializers import EstudianteSerializer


class EstudianteViewSet(viewsets.ModelViewSet[Estudiante]):
    """
    CRUD completo para Estudiante.
    ModelViewSet genera automáticamente: GET /estudiantes/, GET /estudiantes/{id}/,
    POST /estudiantes/, PUT /estudiantes/{id}/, PATCH /estudiantes/{id}/,
    DELETE /estudiantes/{id}/
    """

    queryset = Estudiante.objects.all()
    serializer_class = EstudianteSerializer

    def create(self, request: Request, *args: Any, **kwargs: Any) -> Response:
        """Sobreescribimos create para retornar 201 Created explícitamente."""
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        self.perform_create(serializer)
        return Response(serializer.data, status=status.HTTP_201_CREATED)
```

5.9 Rutas — src/estudiantes/urls.py

El DefaultRouter de DRF genera automáticamente todas las rutas para el ViewSet:

matricula-service/src/estudiantes/urls.py

```
from django.urls import include, path
from rest_framework.routers import DefaultRouter

from .views import EstudianteViewSet

router = DefaultRouter()
router.register("", EstudianteViewSet, basename="estudiante")

urlpatterns = [path("", include(router.urls))]
```

5.10 Replicar lo realizado con cursos

1. Reemplaza el siguiente contenido en el archivo models.py, que ya existe vacío con un comentario:

matricula-service/src/cursos/models.py

```
from django.core.validators import MinValueValidator
from django.db import models

from estudiantes.models import Estudiante
```

```
class Curso(models.Model):
    """Modelo Django para la tabla cursos."""

    codigo = models.CharField(max_length=20, unique=True)
    nombre = models.CharField(max_length=150)
    creditos = models.IntegerField(validators=[MinValueValidator(1)])

    # profesor_id referencia un registro de la tabla 'profesores', que pertenece
    # al profesor-service (FastAPI + SQLAlchemy). No usamos ForeignKey de Django
    # porque el modelo Profesor no existe dentro de este proyecto Django.
    profesor_id = models.IntegerField()

    # Relación muchos a muchos con Estudiante: representa la matrícula.
    # Django crea automáticamente la tabla intermedia cursos_curso_estudiantes.
    estudiantes = models.ManyToManyField(Estudiante, related_name="cursos",
blank=True)

    class Meta:
        db_table = "cursos"
        ordering = ["codigo"]

    def __str__(self) -> str:
        return f"{self.codigo} - {self.nombre}"
```

2. Se debe crear este archivo nuevo (clic derecho sobre cursos → New File → serializers.py). Una vez creado, se debe incluir lo siguiente en ese archivo:

matricula-service/src/cursos/serializer.py

```
from rest_framework import serializers

from .models import Curso

class CursoSerializer(serializers.ModelSerializer[Curso]):
    """Serializer completo para crear, actualizar y leer cursos."""

    class Meta:
        model = Curso
        fields = ["id", "codigo", "nombre", "creditos", "profesor_id",
"estudiantes"]
        # estudiantes (la matrícula) se gestiona con endpoints dedicados, no en el
create/update directo
        read_only_fields = ["estudiantes"]
```

3. Reemplaza el contenido en el archivo views.py que ya fue creado anteriormente:

matricula-service/src/cursos/views.py

```
from typing import Any

from rest_framework import status, viewsets
from rest_framework.request import Request
from rest_framework.response import Response

from .models import Curso
from .serializers import CursoSerializer


class CursoViewSet(viewsets.ModelViewSet[Curso]):
    """CRUD completo para Curso."""

    queryset = Curso.objects.all()
```

```
    serializer_class = CursoSerializer

    def create(self, request: Request, *args: Any, **kwargs: Any) -> Response:
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        self.perform_create(serializer)
        return Response(serializer.data, status=status.HTTP_201_CREATED)
```

4. matricula-service/src/cursos/urls.py
Crea este archivo nuevo (clic derecho sobre cursos → New File → urls.py):

matricula-service/src/cursos/urls.py

```
from django.urls import include, path
from rest_framework.routers import DefaultRouter

from .views import CursoViewSet

router = DefaultRouter()
router.register("", CursoViewSet, basename="curso")

urlpatterns = [path("", include(router.urls))]
```

5.11 Crear y ejecutar las migraciones

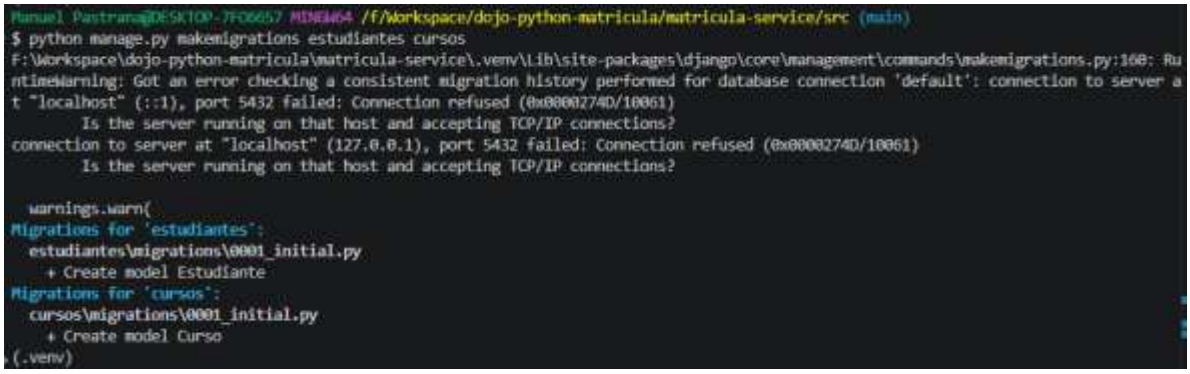
Las migraciones de Django son el equivalente a los archivos SQL de Flyway. Django las genera automáticamente a partir del código de los modelos:

```
cd matricula-service/src

# Generar archivos de migración a partir de los modelos
# (equivalente a crear V001__create_tables.sql en Flyway)
python manage.py makemigrations estudiantes cursosV

# Aplicar las migraciones a la base de datos
python manage.py migrate

# Resultado esperado:
```



> ⚠ *Las migraciones requieren que PostgreSQL esté corriendo. Si no tienes PostgreSQL local, configura Docker Compose (Paso 6) y ejecuta 'docker compose up -d postgres' antes de hacer migrate. Por el momento levantar un contenedor como se hizo para estudiantes previamente. Con eso podemos probar por el momento mientras se llega al paso 6.*

```
docker run -d --name postgres-matricula-dev \
  -e POSTGRES_DB=matricula_db \
  -e POSTGRES_USER=matricula_user \
  -e POSTGRES_PASSWORD=matricula_pass \
  -p 5433:5432 \
  postgres:16-alpine
```

5.12 Test de los servicios

A continuación, se presenta resumido los pasos seguidos para crear las pruebas:

En profesor-service, "unitario" significaba *SQLite en memoria* (sin Docker) vs "integración" significaba *PostgreSQL real vía Testcontainers*. Esa frontera era posible porque SQLAlchemy te deja intercambiar el motor de base de datos libremente. En Django, el ORM está **acoplado a un único backend configurado** en DATABASES (PostgreSQL en nuestro caso). pytest-django no nos da gratis un "modo SQLite" para tests y aunque se podría forzar con una configuración de settings separada, agregaría una complejidad considerable para un dojo académico, y no es la práctica más común en proyectos Django reales.

La distinción que SÍ tiene sentido en Django:

	Unitario	Integración
Qué prueba	El Serializer directamente — reglas de validación (validate_semestre, formato de email, unicidad)	El endpoint HTTP completo: routing → ViewSet → Serializer → ORM → respuesta JSON
Cómo se invoca	EstudianteSerializer(data=...) — llamada directa a Python	api_client.post("/api/v1/estudiantes", ...) — petición HTTP real
¿Usa BD?	Sí (los UniqueValidator de los campos unique=True consultan la BD aunque no hagas HTTP)	Sí
Qué detecta	Errores de reglas de negocio/validación	Errores de routing, serialización, códigos de estado, contrato HTTP

Es decir: **ambos usan PostgreSQL** (esa es la limitación que presenta Django), pero la separación por **qué capa se está probando** (serializer aislado vs. API completa) sigue siendo válida y útil. En otras palabras, es la misma idea que @WebMvcTest vs @SpringBootTest en Java, donde ambos pueden usar BD pero prueban capas distintas.

Estructura de carpetas:

```
matricula-service/tests/
├── __init__.py
├── conftest.py                          # fixture api_client, compartido
├── unit/
│   ├── __init__.py
│   └── test_estudiantes_serializer.py   # valida el Serializer directamente
└── integration/
    ├── __init__.py
    └── test_estudiantes_api.py          # valida los endpoints HTTP completos
```

1. crear en matricula-service/test la estructura de carpetas mencionada en la imagen anterior. Esto quiere decir, que debemos crear la carpeta unit, la carpeta test. Realizado lo anterior, se debe crear dentro de cada carpeta el respectivo __init__.py vacío. También a nivel de la carpeta test, se debe crear el __init__.py vacío.

2. Crear el archivo conftest.py en la ruta matricula-service/tests y remplazar el contenido por:

Matricula-service/test/conftest.py

```
import pytest
from rest_framework.test import APIClient

@pytest.fixture
def api_client() -> APIClient:
    """Cliente HTTP de pruebas para los endpoints REST de Django."""
    return APIClient()
```

Nota importante sobre aislamiento: a diferencia de profesor-service (donde tuvimos que crear manualmente el fixture limpiar_bd), pytest-django envuelve **cada test individual** en una transacción que se revierte automáticamente al terminar — gracias al decorador @pytest.mark.django_db que verás en cada clase de test. No necesitamos limpiar la BD manualmente.

3. Dentro de la carpeta de unit crear el archivo test_estudiantes_serializer.py pegar el siguiente contenido.

Matricula-service/test/unit/test_estudiantes_serializer.py

```
import pytest

from estudiantes.serializers import EstudianteSerializer

VALID_DATA = {
    "nombre": "Laura",
    "apellido": "Gomez",
    "email": "laura.gomez@correounivalle.edu.co",
    "identificacion": "1144556677",
    "semestre": 5,
}

@pytest.mark.django_db
class TestEstudianteSerializer:
    """
    Tests unitarios: validan la lógica del Serializer directamente,
    sin pasar por HTTP ni por el ViewSet.

    Nota: usan @pytest.mark.django_db porque los campos unique=True
    (email, identificacion) generan un UniqueValidator que consulta la BD.
    """

    def test_datos_validos_es_valido(self) -> None:
        serializer = EstudianteSerializer(data=VALID_DATA)
        assert serializer.is_valid()

    def test_semestre_menor_a_uno_no_es_valido(self) -> None:
        data = {**VALID_DATA, "identificacion": "0000000001", "email":
"a@correounivalle.edu.co", "semestre": 0}
        serializer = EstudianteSerializer(data=data)
        assert not serializer.is_valid()
        assert "semestre" in serializer.errors

    def test_semestre_mayor_a_diez_no_es_valido(self) -> None:
        data = {**VALID_DATA, "identificacion": "0000000002", "email":
"b@correounivalle.edu.co", "semestre": 11}
```

```
        serializer = EstudianteSerializer(data=data)
        assert not serializer.is_valid()
        assert "semestre" in serializer.errors

    def test_email_con_formato_invalido_no_es_valido(self) -> None:
        data = {**VALID_DATA, "identificacion": "0000000003", "email": "no-es-un-
email"}
        serializer = EstudianteSerializer(data=data)
        assert not serializer.is_valid()
        assert "email" in serializer.errors
```

4. Dentro de la carpeta de unit crear el archivo test_estudiantes_api.py y pegar el siguiente contenido:

Matricula-service/test/unit/test_estudiantes_serializer.py

```
import pytest
from rest_framework.test import APIClient

DATA = {
    "nombre": "Laura",
    "apellido": "Gomez",
    "email": "laura.gomez@correounivalle.edu.co",
    "identificacion": "1144556677",
    "semestre": 5,
}

@pytest.mark.django_db
class TestEstudianteAPI:
    def test_crear_estudiante_retorna_201(self, api_client: APIClient) -> None:
        r = api_client.post("/api/v1/estudiantes/", DATA, format="json")
        assert r.status_code == 201
        assert r.data["email"] == DATA["email"]

    def test_crear_email_duplicado_retorna_400(self, api_client: APIClient) ->
None:
        api_client.post("/api/v1/estudiantes/", DATA, format="json")
        r = api_client.post("/api/v1/estudiantes/", DATA, format="json")
        assert r.status_code == 400

    def test_listar_estudiantes_retorna_200(self, api_client: APIClient) -> None:
        r = api_client.get("/api/v1/estudiantes/")
        assert r.status_code == 200
        assert isinstance(r.data, list)

    def test_obtener_por_id_retorna_200(self, api_client: APIClient) -> None:
        creado = api_client.post("/api/v1/estudiantes/", DATA, format="json").data
        r = api_client.get(f"/api/v1/estudiantes/{creado['id']}/")
        assert r.status_code == 200

    def test_obtener_id_inexistente_retorna_404(self, api_client: APIClient) ->
None:
        r = api_client.get("/api/v1/estudiantes/999999/")
        assert r.status_code == 404

    def test_actualizar_retorna_200_con_datos_nuevos(self, api_client: APIClient) -
> None:
        creado = api_client.post("/api/v1/estudiantes/", DATA, format="json").data
        r = api_client.put(
            f"/api/v1/estudiantes/{creado['id']}/",
            {**DATA, "nombre": "Laura Actualizada"},
            format="json",
        )
```

```
        assert r.status_code == 200
        assert r.data["nombre"] == "Laura Actualizada"

    def test_eliminar_retorna_204(self, api_client: APIClient) -> None:
        creado = api_client.post("/api/v1/estudiantes/", DATA, format="json").data
        r = api_client.delete(f"/api/v1/estudiantes/{creado['id']}/")
        assert r.status_code == 204

    def test_crear_con_semestre_invalido_retorna_400(self, api_client: APIClient) -
> None:
        datos_invalidos = {
            **DATA,
            "identificacion": "0000000000",
            "email": "invalido@correounivalle.edu.co",
            "semestre": 11,
        }
        r = api_client.post("/api/v1/estudiantes/", datos_invalidos, format="json")
        assert r.status_code == 400
```

Se recomienda revisar en detalle los casos de prueba que contienen cada una para comprender todo lo que se logra cubrir. A medida que el proyecto sigue evolucionando, estas pruebas incluirán más casos.

Finalmente, verificamos que los test corren y la cobertura es superior al 80%:

```
# Todos los tests con reporte de cobertura en terminal
pytest --cov=src --cov-report=term-missing
```

```
$ pytest --cov=src --cov-report=term-missing
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html

---------- coverage: platform win32, python 3.13.2-final-0 -----------
Name                              Stmts   Miss  Cover   Missing
---------------------------------------------------------------
src\config\__init__.py                0      0   100%
src\config\asgi.py                    4      4     0%   10-16
src\config\settings.py               20      0   100%
src\config\urls.py                    4      0   100%
src\config\views.py                   3      1    67%   6
src\config\wsgi.py                    4      4     0%   10-16
src\cursos\__init__.py                0      0   100%
src\cursos\admin.py                   0      0   100%
src\cursos\apps.py                    4      0   100%
src\cursos\models.py                 14      1    93%   28
src\cursos\serializers.py             7      0   100%
src\cursos\tests.py                   0      0   100%
src\cursos\urls.py                    6      0   100%
src\cursos\views.py                  14      4    71%   18-21
src\estudiantes\__init__.py           0      0   100%
src\estudiantes\admin.py              0      0   100%
src\estudiantes\apps.py               4      0   100%
src\estudiantes\models.py            13      1    92%   22
src\estudiantes\serializers.py       10      1    90%   19
src\estudiantes\tests.py              0      0   100%
src\estudiantes\urls.py               6      0   100%
src\estudiantes\views.py             14      0   100%
---------------------------------------------------------------
TOTAL                               127     16    87%
```

Paso 6 — Configuración del servidor EC2

El proceso es idéntico al del dojo Java. Si ya tienes el servidor configurado, solo necesitas subir los nuevos archivos de despliegue, pero la recomendación es no hacerlo sobre la misma carpeta app que se hizo en el otro dojo.

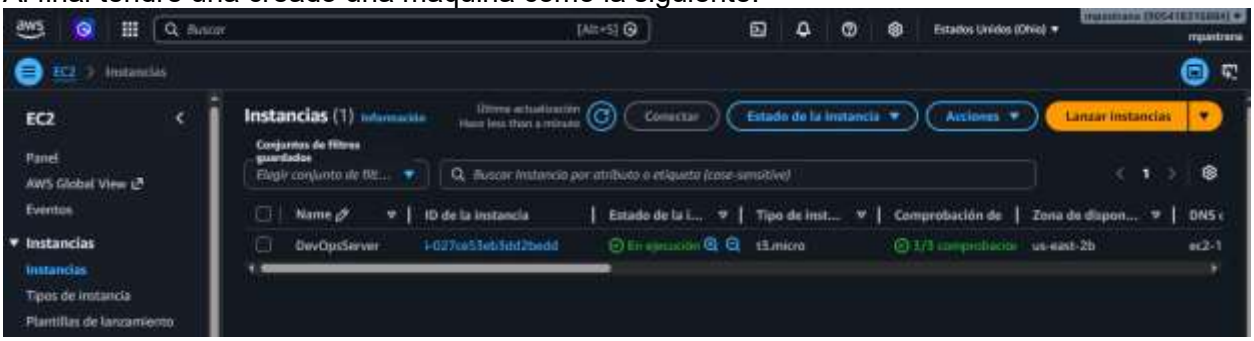
6.1 Crear la instancia EC2

Accede a https://aws.amazon.com → EC2 → Launch instances y configura:

Campo	Valor
Name	demo-ds-univalle-prod
OS Image	Amazon Linux 2023 AMI
Instance type	t3.micro (capa gratuita)
Key pair	Crear nuevo: demo-ds-univalle-key (RSA, .pem)
Auto-assign public IP	Enable

> *⚠ Guarda el archivo .pem en un lugar seguro. No lo subas a GitHub. Sin él no puedes conectarte al EC2.*

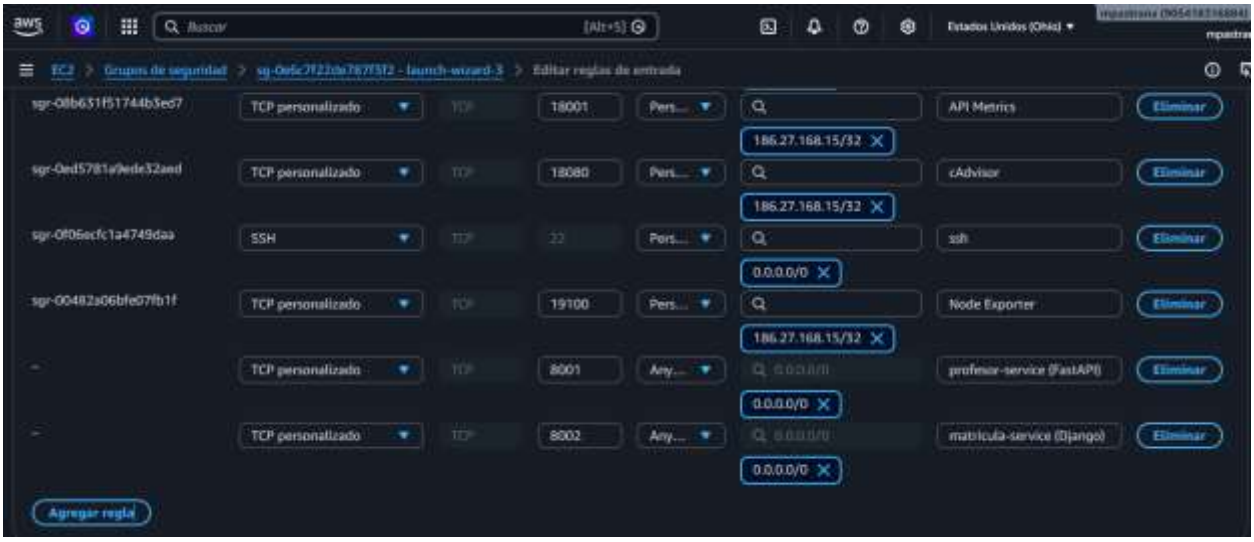
Al final tendré una creado una máquina como la siguiente:



Es importante resaltar que mi caso cree la máquina con otro nombre, pero sirve como visualización del ejemplo de una instancia corriendo en AWS.

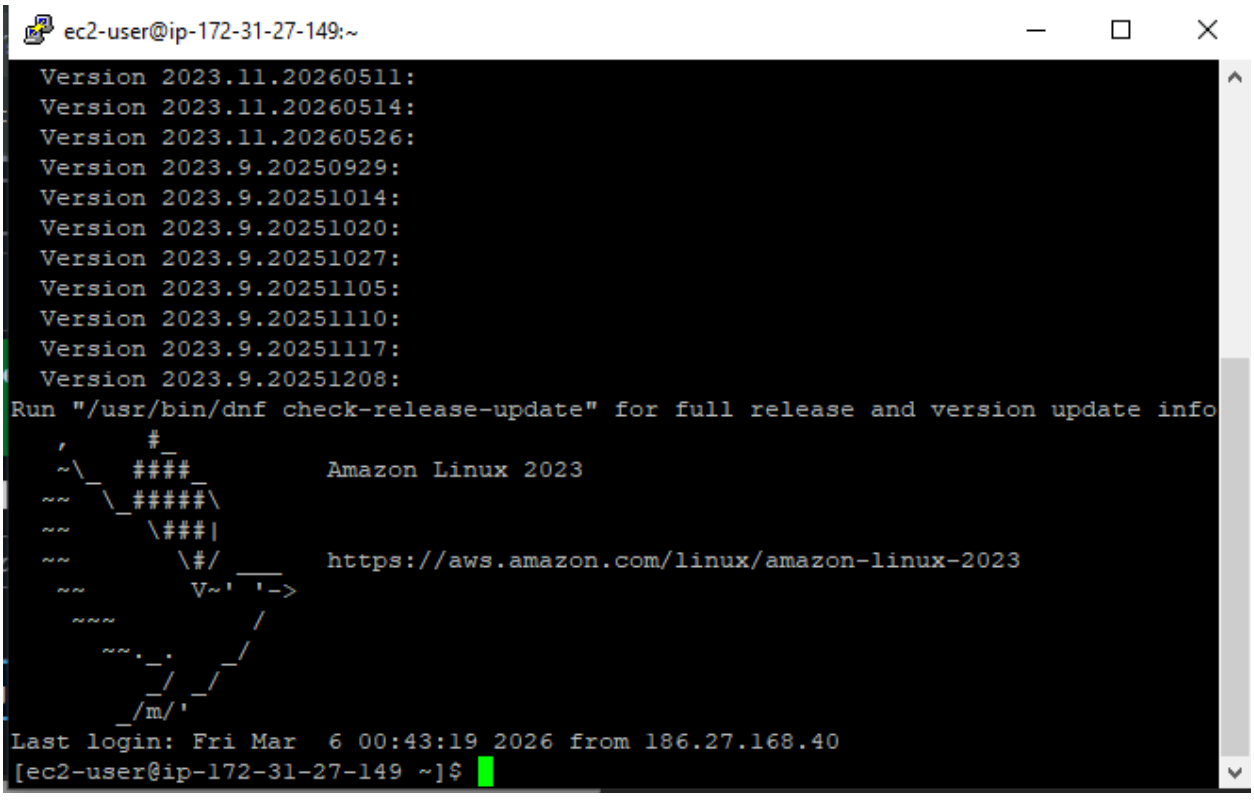
6.2 Security Group — puertos a abrir

Type	Protocol	Port	Source	Descripción
SSH	TCP	22	My IP	Acceso SSH solo desde tu IP
Custom TCP	TCP	8001	0.0.0.0/0	profesor-service (FastAPI)
Custom TCP	TCP	8002	0.0.0.0/0	matricula-service (Django)



6.3 Conectarse al EC2

```
chmod 400 demo-ds-univalle-key.pem
ssh -i demo-ds-univalle-key.pem ec2-user@TU_IP_PUBLICA
```



6.4 Instalar Docker en el EC2

```
sudo dnf update -y
sudo dnf install -y docker
sudo systemctl start docker
```

```
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
newgrp docker
docker --version
docker run hello-world
```

6.5 Instalar Docker Compose

```
sudo mkdir -p /usr/local/lib/docker/cli-plugins


sudo curl -SL
https://github.com/docker/compose/releases/download/v2.27.0/docker-compose-
linux-x86_64 \
  -o /usr/local/lib/docker/cli-plugins/docker-compose

sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
docker compose version
```

6.6 Crear directorios y archivo .env en el EC2

```
mkdir -p ~/dojo_python
nano ~/dojo_python/.env
```

Contenido del .env:

```
DB_NAME=matricula_db
DB_USER=matricula_user
DB_PASS=tu_password_seguro_aqui
DJANGO_SECRET_KEY=clave-secreta-larga-y-aleatoria-para-django
DOCKERHUB_USERNAME=mapastrana
```

> *DB_HOST=postgres: dentro de la red de Docker Compose los contenedores se comunican por nombre de servicio, no por IP.*

Para generar el DB_PASS, se puede recurrir al siguiente comando en consola en su máquina openssl rand -hex 24. Eso retornará un valor que pueden pegar ahí. Para el DJANGO_SECRET_KEY pueden usar el comando python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"

Es importante también habilitar que github pueda conectarse al EC2 por ssh-rsa. Activa en el EC2 esto para poder conectarte. El siguiente comando lo hace

```
echo "PubkeyAcceptedAlgorithms +ssh-rsa" | sudo tee
/etc/ssh/sshd_config.d/allow-rsa.conf
sudo systemctl restart sshd
```

Paso 7 — Docker y Docker Compose

Dockerizamos ambos microservicios para garantizar que el comportamiento sea idéntico en desarrollo local, en CI y en producción.

7.1 Dockerfile del profesor-service

Usamos un Dockerfile multietapa para reducir el tamaño de la imagen final: la primera etapa instala las dependencias y la segunda solo copia lo necesario para ejecutar.

profesor-service/Dockerfile

```
# ── Etapa 1: construcción ──────────────────────────────────────────
FROM python:3.12-slim AS builder
WORKDIR /app
COPY pyproject.toml .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -e .

# ── Etapa 2: imagen final de producción ────────────────────────────
FROM python:3.12-slim
WORKDIR /app

# Copiar solo los paquetes instalados (no el código fuente de pip ni las
herramientas de dev)
COPY --from=builder /usr/local/lib/python3.12 /usr/local/lib/python3.12
COPY --from=builder /usr/local/bin /usr/local/bin

# Copiar el código fuente
COPY src/ ./src/

# Variables de entorno necesarias para Python
ENV PYTHONPATH=/app/src
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

EXPOSE 8001

# Comando de inicio: uvicorn es el servidor ASGI de FastAPI
CMD ["uvicorn", "profesor.main:app", "--host", "0.0.0.0", "--port", "8001"]
```

7.2 Dockerfile del matricula-service

matricula-service/Dockerfile

```
FROM python:3.12-slim AS builder
WORKDIR /app
COPY pyproject.toml .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -e .

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.12 /usr/local/lib/python3.12
COPY --from=builder /usr/local/bin /usr/local/bin
COPY src/ ./src/

ENV PYTHONPATH=/app/src
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV DJANGO_SETTINGS_MODULE=config.settings
```

```
EXPOSE 8002

# Ejecutar migraciones y luego iniciar el servidor Django
CMD ["sh", "-c", "python src/manage.py migrate && python src/manage.py runserver
0.0.0.0:8002"]
```

7.3 docker-compose.yml — entorno de desarrollo local

Este archivo levanta los tres servicios necesarios para desarrollo: PostgreSQL, profesor-service y matricula-service. Colócalo en la raíz del repositorio.

docker-compose.yml

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: matricula_postgres_dev
    environment:
      POSTGRES_DB: matricula_db
      POSTGRES_USER: matricula_user
      POSTGRES_PASSWORD: matricula_pass
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U matricula_user -d matricula_db"]
      interval: 10s
      timeout: 5s
      retries: 5

  profesor-service:
    build: ./profesor-service
    container_name: profesor_service_dev
    ports:
      - "8001:8001"
    environment:
      -
DATABASE_URL=postgresql://matricula_user:matricula_pass@postgres:5432/matricula_db
    depends_on:
      postgres:
        condition: service_healthy

  matricula-service:
    build: ./matricula-service
    container_name: matricula_service_dev
    ports:
      - "8002:8002"
    environment:
      -
DATABASE_URL=postgresql://matricula_user:matricula_pass@postgres:5432/matricula_db
      - SECRET_KEY=dev-secret-key-cambiar-en-produccion
    depends_on:
      postgres:
        condition: service_healthy

volumes:
  postgres_data:
```

A diferencia del dojo Java (un solo demo-ds-univalle), aquí tienes **dos imágenes independientes** que construir y subir por separado en dockerhub.

Imagen profesor-service

```
docker build -t TU_USUARIO/profesor-service:latest ./profesor-service
```

Imagen matricula-service

```
docker build -t TU_USUARIO/matricula-service:latest ./matricula-service
```

Reemplaza TU_USUARIO con tu nombre de usuario de Docker Hub. El punto al final indica que el Dockerfile está en el directorio actual. Recuerden que si docker no esta corriendo en su máquina estos comandos van a fallar.

3. Verificar que la imagen fue creada:

```
docker images | grep TU_USUARIO(si la consola es unix)
```

4. Subir la imagen a docker Hub

Primero, se debe iniciar sesión con el siguiente comando. No olviden ingresar Usuario y password cuando se los pida

```
docker login
```

Luego se debe subir al docker hub usando el siguiente comando:

```
docker push TU_USUARIO/demo-ds-univalle:latest
docker push TU_USUARIO/profesor-service:latest
docker push TU_USUARIO/matricula-service:latest
```

7.4 Crear el docker-compose.prod.yml en el EC2

Este archivo se crea en la carpeta raíz del proyecto. El Contenido es el siguiente:

docker-compose.prod.yml

```
services:

  postgres:
    image: postgres:16-alpine
    container_name: matricula_python_prod_postgres
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASS}
    volumes:
      - matricula_prod_data:/var/lib/postgresql/data
    networks:
      - dojo_python_network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped

  profesor-service:
    image: ${DOCKERHUB_USERNAME}/profesor-service:latest
    container_name: profesor_service_prod
    environment:
      DATABASE_URL: postgresql://${DB_USER}:${DB_PASS}@postgres:5432/${DB_NAME}
    ports:
      - "8001:8001"
    depends_on:
      postgres:
        condition: service_healthy
    networks:
      - dojo_python_network
    restart: unless-stopped
```

```
  matricula-service:
    image: ${DOCKERHUB_USERNAME}/matricula-service:latest
    container_name: matricula_service_prod
    environment:
      DATABASE_URL: postgresql://${DB_USER}:${DB_PASS}@postgres:5432/${DB_NAME}
      SECRET_KEY: ${DJANGO_SECRET_KEY}
      DEBUG: "False"
    ports:
      - "8002:8002"
    depends_on:
      postgres:
        condition: service_healthy
    networks:
      - dojo_python_network
    restart: unless-stopped

networks:
  dojo_python_network:
    driver: bridge

volumes:
  matricula_prod_data:
```

Cuando ya lo temenos construido, lo debemos enviar a la carpeta dojo_python en el EC2. Para subirlo_

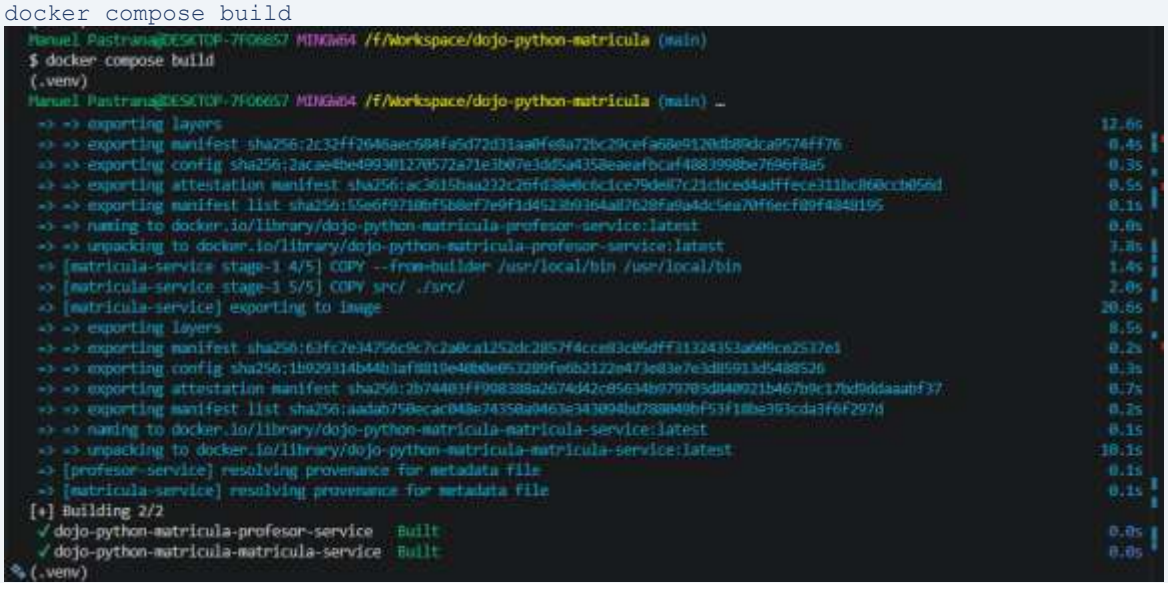
```
# Desde Git Bash en Windows
pscp -i /ruta/a/tu/clave.ppk docker-compose.prod.yml ec2-user@TU_IP:/home/ec2-
user/dojo_python/

# Desde Mac o Linux
scp -i /ruta/a/tu/clave.pem docker-compose.prod.yml ec2-user@TU_IP:/home/ec2-
user/dojo_python/
```

7.4 Levantar el stack y verificar

Desde la raíz del repositorio:

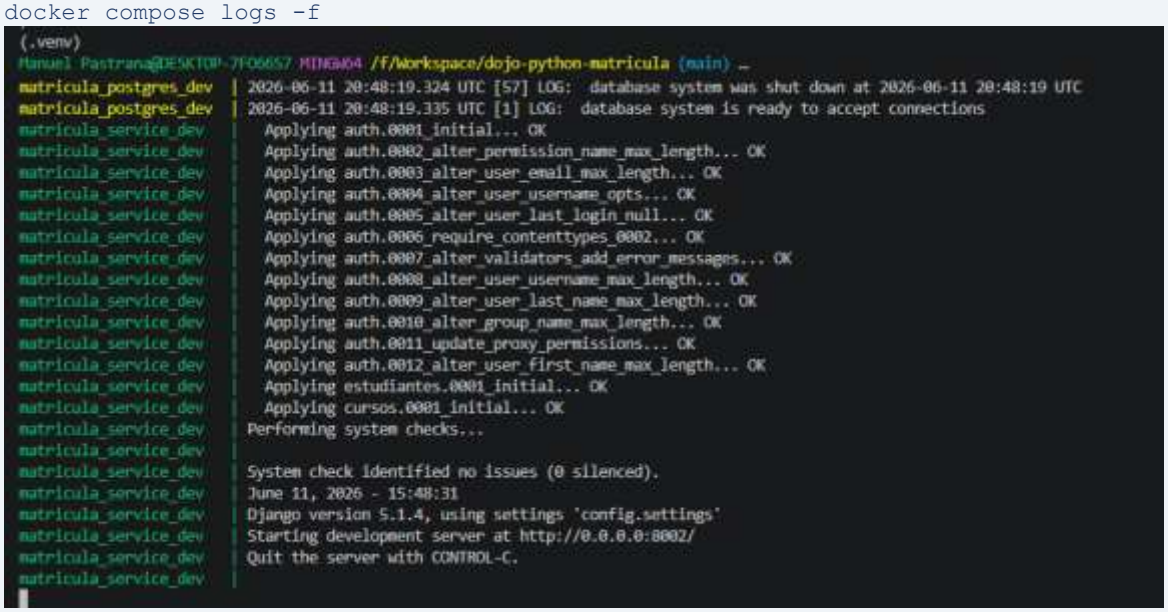
```
# Construir las imágenes Docker de ambos servicios
docker compose build
```



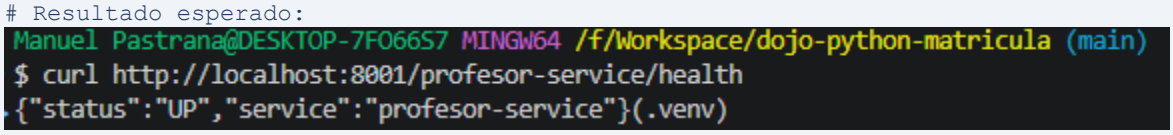
```
# Levantar todos los servicios en segundo plano
docker compose up -d
```



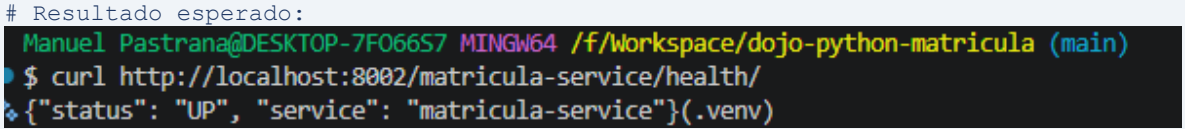

```
# Ver los logs en tiempo real
docker compose logs -f
```




```
# Verificar que los servicios responden
curl http://localhost:8001/profesor-service/health
# Resultado esperado:
```




```
curl http://localhost:8002/matricula-service/health/
# Resultado esperado:
```

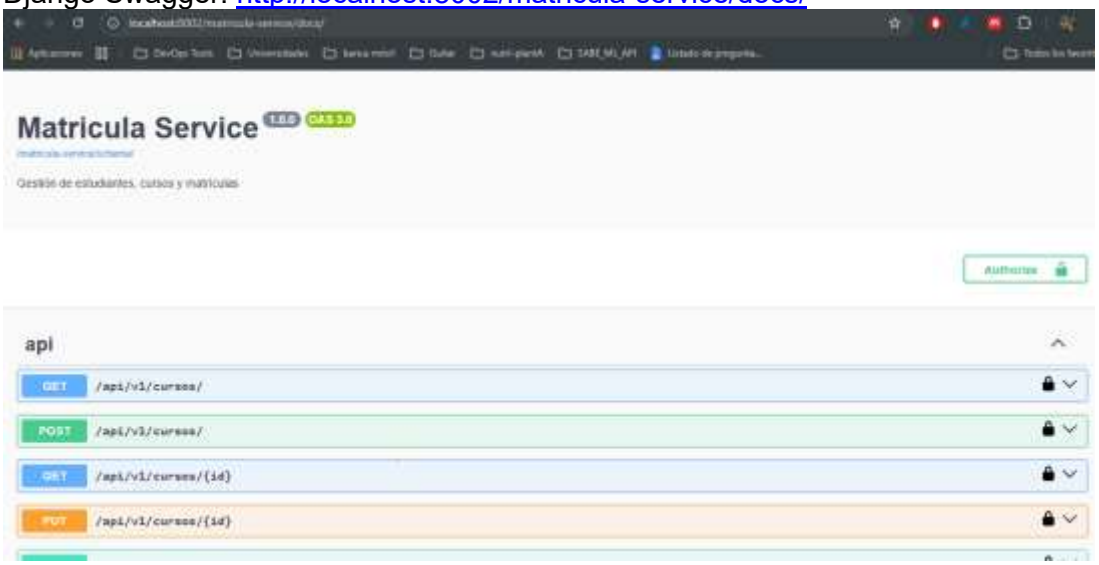


Abre en el navegador:

- FastAPI Swagger: http://localhost:8001/profesor-service/docs



- Django Swagger: http://localhost:8002/matricula-service/docs/



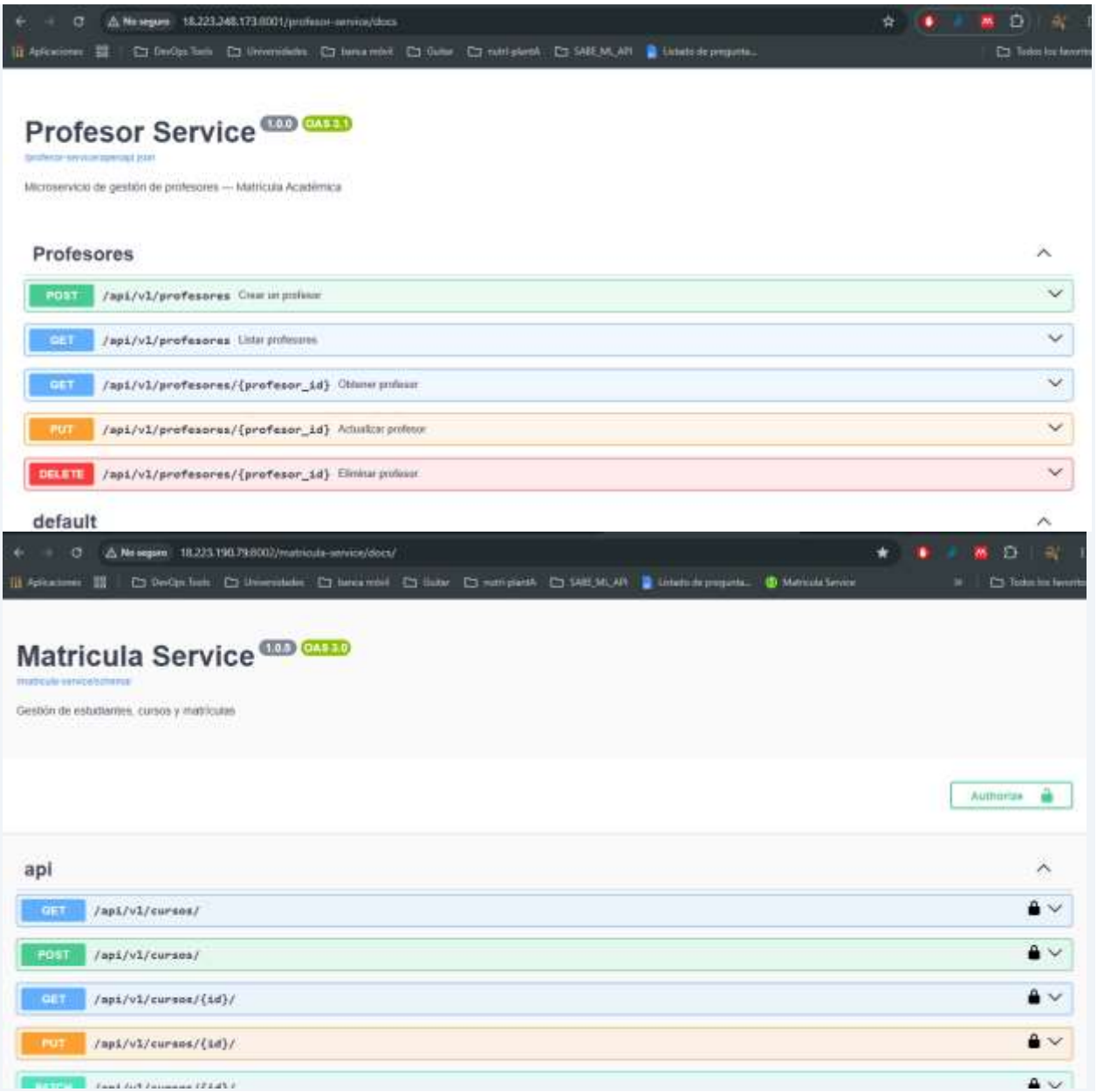

```
# Detener todos los servicios
docker compose down

# Detener y eliminar volúmenes (borra los datos de la BD)
docker compose down -v
```

Si todo salió bien localmente, ahora debemos comprobar si sale de la misma forma en el EC2. Conectado a la instancia ejecutar el siguiente comando dentro de la carpeta dojo_python.

```
docker compose -f docker-compose.prod.yml up -d
```

Si todo está bien deberías poder ver los swagger de ambos servicios.



Paso 8 — Pipeline CI/CD con GitHub Actions

El pipeline tiene la misma estructura que el dojo Java: CI corre en cada PR hacia develop; CD corre cuando se hace merge a main.

Recuerden antes de hacer el pipeline, ejecuten ruff, mypy y pyptests primero, para dejar el código limpio.

```
# Verificar y corregir el formato automáticamente
ruff format src tests
```

```
# Verificar errores de linting y corregir automáticamente
ruff check src tests --fix
# Resultado esperado: All checks passed.
```

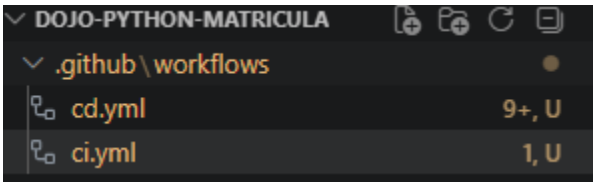
```
# Verificar tipos estáticos
mypy src
# Resultado esperado: Success: no issues found in N source files

# Si mypy muestra errores falsos después de cambios, limpia la caché:
rm -rf .mypy_cache
mypy src
```

Estoy seguro de que saldrán algunas cosas con ruff, es algo normal del desarrollo. Como ejercicio propuesto corregir lo que les salga.

8.1 Pipeline CI — .github/workflows/ci.yml

El CI valida ambos microservicios de forma paralela. Para esto, va a ejecutar ruff, mypy y pytest en cada uno tal y como lo hemos realizado en nuestro entorno local. Al terminar, SonarCloud analiza la cobertura. Recuerden que para ejecutar el git Actions, git hub necesita que se le comparta la configuración de pasos a seguir. Para eso vamos a crear el archivo ci.yml dentro de la carpeta workflows de .github en el proyecto.



Ya creado el archivo los invito a pegar las siguientes instrucciones y analizar su contenido.

.github/workflows/ci.yml

```
name: CI — Integración Continua

on:
  pull_request:
    branches:
      - develop

permissions:
  contents: read
  pull-requests: write

jobs:

  # ── Job 1: validar el profesor-service (FastAPI) ──────────────────────
  profesor-service:
    name: profesor-service — ruff / mypy / pytest
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: profesor-service  # Todos los comandos desde esta
carpeta
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - name: Instalar dependencias de desarrollo
        run: pip install -e '.[dev]'
```

```
      - name: ruff lint — verificar errores de código
        run: ruff check src tests

      - name: ruff format — verificar formato
        run: ruff format --check src tests

      - name: mypy — verificar tipos estáticos
        run: mypy src

      - name: pytest unitarios — tests sin Docker
        run: pytest tests/unit -v --cov=src --cov-report=xml

      - name: pytest integración — tests con PostgreSQL real
        run: pytest tests/integration -v

      - name: Verificar cobertura >= 80%
        run: pytest --cov=src --cov-fail-under=80

      - name: Subir reporte de cobertura como artefacto
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: coverage-profesor
          path: profesor-service/coverage.xml

  # ── Job 2: validar el matricula-service (Django) ──────────────────────
  matricula-service:
    name: matricula-service — ruff / mypy / pytest
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: matricula-service
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_DB: matricula_db
          POSTGRES_USER: matricula_user
          POSTGRES_PASSWORD: matricula_pass
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - name: Instalar dependencias de desarrollo
        run: pip install -e '.[dev]'

      - name: ruff lint
        run: ruff check src tests

      - name: ruff format check
        run: ruff format --check src tests
```

```
      - name: mypy
        run: mypy src

      - name: pytest con cobertura
        env:
          DJANGO_SETTINGS_MODULE: config.settings
        run: pytest --cov=src --cov-report=xml --cov-fail-under=80

      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: coverage-matricula
          path: matricula-service/coverage.xml

  # ── Job 3: SonarCloud (solo si CI pasa) ──────────────────────────────
  sonar:
    name: SonarCloud — Análisis de calidad
    needs: [profesor-service, matricula-service]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: actions/download-artifact@v4
        with:
          name: coverage-profesor
          path: profesor-service/

      - uses: actions/download-artifact@v4
        with:
          name: coverage-matricula
          path: matricula-service/

      - uses: SonarSource/sonarqube-scan-action@master
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: https://sonarcloud.io
```

Es muy importante recordar que django a diferencia de fastapi requiere tener un contenedor de la base de datos para poder hacer las pruebas. Por eso, en la configuración del segundo job, se ve que utiliza la misma conexión que habíamos probado antes en local.

8.2 Pipeline CD — .github/workflows/cd.yml

El CD construye las imágenes Docker de ambos servicios, las sube a Docker Hub y las despliega en el servidor EC2 vía SSH. Se crea el archivo cd.yml en el mismo lugar que se creo el ci.yml.

.github/workflows/cd.yml

```
name: CD — Despliegue Continuo

on:
  push:
    branches:
      - main   # Se dispara cuando se hace merge a main

jobs:
  cd:
    name: Build, Push y Despliegue en EC2
    runs-on: ubuntu-latest
```

```
    steps:

      - uses: actions/checkout@v4

      - name: Login en Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - uses: docker/setup-buildx-action@v3

      - name: Build y Push — profesor-service
        uses: docker/build-push-action@v5
        with:
          context: ./profesor-service
          push: true
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/profesor-service:latest
            ${{ secrets.DOCKERHUB_USERNAME }}/profesor-service:${{ github.sha }}

      - name: Build y Push — matricula-service
        uses: docker/build-push-action@v5
        with:
          context: ./matricula-service
          push: true
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/matricula-service:latest
            ${{ secrets.DOCKERHUB_USERNAME }}/matricula-service:${{ github.sha }}

      - name: Desplegar en EC2 via SSH
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${{ secrets.EC2_HOST }}
          username: ${{ secrets.EC2_USER }}
          key: ${{ secrets.EC2_SSH_KEY }}
          script: |
            cd ~/dojo_python
            docker pull ${{ secrets.DOCKERHUB_USERNAME }}/profesor-service:latest
            docker pull ${{ secrets.DOCKERHUB_USERNAME }}/matricula-service:latest
            docker compose -f docker-compose.prod.yml down --remove-orphans || true
            docker compose -f docker-compose.prod.yml up -d
            sleep 20
            curl -f http://localhost:8001/profesor-service/health && echo
'profesor-service UP'
            curl -f http://localhost:8002/matricula-service/health/ && echo
'matricula-service UP'
```

8.3 Secrets de GitHub requeridos

Ve a tu repositorio en GitHub → Settings → Secrets and variables → Actions → New repository secret y agrega:

Secret	Valor
DOCKERHUB_USERNAME	Tu nombre de usuario en Docker Hub
DOCKERHUB_TOKEN	Personal Access Token de Docker Hub con permisos Read & Write

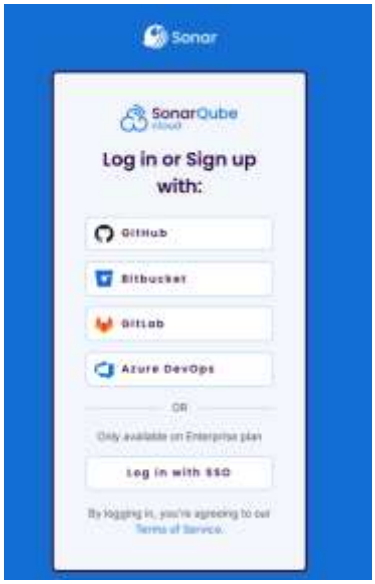
Secret	Valor
EC2_HOST	IP pública de tu servidor EC2 (ej: 18.223.248.173)
EC2_USER	ec2-user
EC2_SSH_KEY	Contenido completo de tu archivo .pem (incluyendo -----BEGIN y -----END RSA PRIVATE KEY-----)
SONAR_TOKEN	Token generado en SonarCloud: My Account → Access Tokens → Generate

Paso 9 — SonarCloud

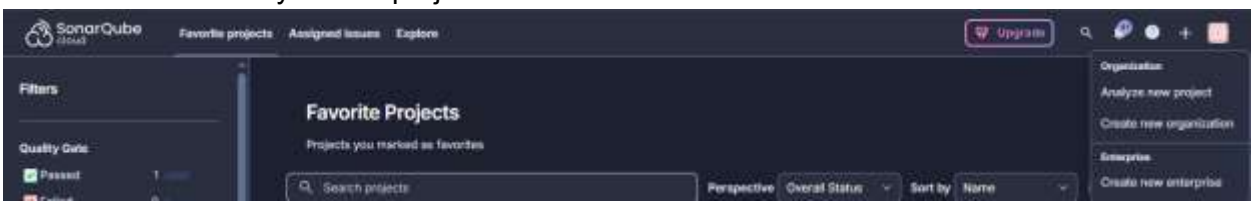
9.1 Activar el proyecto en SonarCloud

Antes de crear el archivo de configuración, necesitamos que SonarCloud genere los valores específicos de este proyecto (projectKey, organization y el token de acceso). Sigue exactamente los mismos pasos que en el dojo Java:

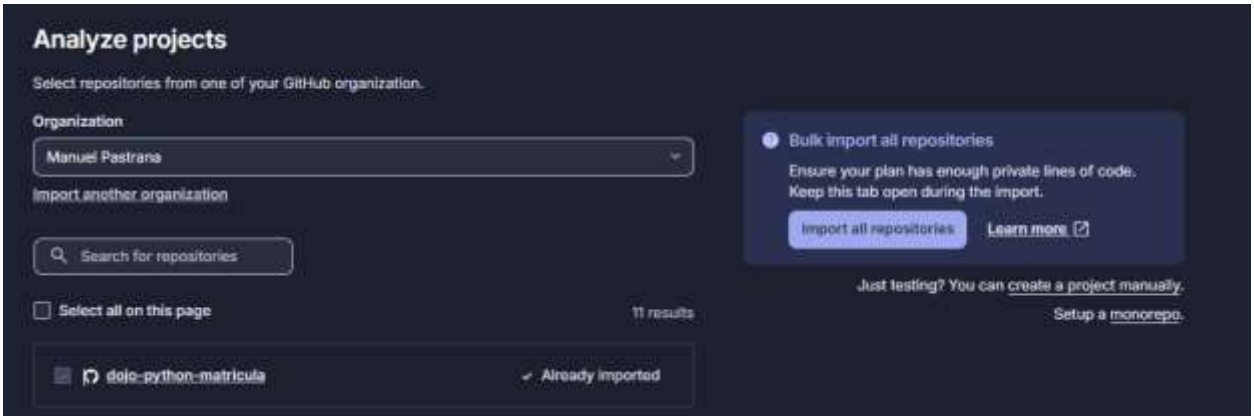
- Ve a sonarcloud.io e inicia sesión con tu cuenta de GitHub



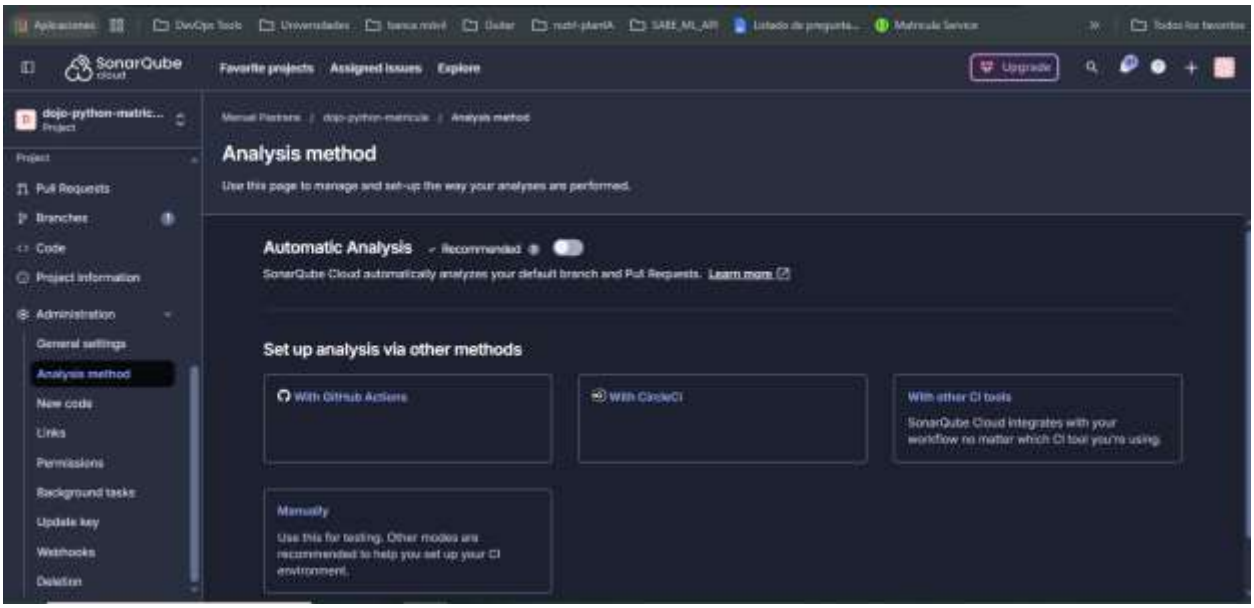
- Haz clic en + → Analyze new project



- Selecciona el repositorio dojo-python-matricula



- Selecciona With GitHub Actions (si no te aparece, después de creado el proyecto ve a administración -> método de análisis



- Copia el SONAR_TOKEN y agrégalo como secret en GitHub
- IMPORTANTE: Desactiva el Automatic Analysis en Administration → Analysis Method → desactiva el toggle

> ⚠ *Si Automatic Analysis queda activo al mismo tiempo que el pipeline CI, SonarCloud rechazará el análisis del CI con un error. Siempre desactívalo cuando uses GitHub Actions. Recuerda tambien tomarlos datos de projectKey y organization. El sonar token será usado en los secrets de github.*

9.2 sonar-project.properties

Sonar requiere del identificador del proyecto, la organización y la orientación de que puede escanerar. Para esto es necesario crea el archivo sonar-project.properties en la raíz del repositorio. SonarCloud lo usa para saber qué analizar:

sonar-project.properties

```
sonar.projectKey=TU_USUARIO_dojo-python-matricula
sonar.organization=TU_USUARIO
sonar.host.url=https://sonarcloud.io

# Rutas de los reportes de cobertura de cada servicio
```

```
sonar.python.coverage.reportPaths=profesor-service/coverage.xml,matricula-
service/coverage.xml

# Excluir archivos que no son código de negocio
sonar.exclusions=**/migrations/**,**/__pycache__/**,**/alembic/**,**/tests/**
```

Paso 10 — Flujo de trabajo Git

10.1 Ramas del proyecto

Rama	Propósito	Recibe PRs desde
main	Código en producción. Cada merge dispara el CD.	Solo desde develop
develop	Rama de integración. Acumula los cambios listos.	Ramas personales (mpastrana)
mpastrana	Rama de trabajo del profesor. Todo cambio sale de aquí.	—

10.2 Crear las ramas

```
# Crear y publicar la rama develop
git checkout -b develop
git push origin develop

# Crear y publicar la rama personal
git checkout -b mpastrana
git push origin mpastrana
```

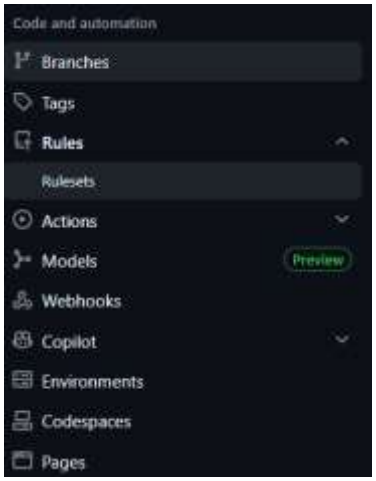
10.3 Proteger las ramas en GitHub

Ve a tu repositorio → Settings → Rules → Rulesets → Add branch ruleset. Crea dos reglas:

Ruleset	Target branch	Reglas a activar
protect-main	main	Restrict deletions + Require a pull request before merging (approvals: 0) + Block force pushes
protect-develop	develop	Restrict deletions + Require a pull request before merging (approvals: 0) + Block force pushes

> *⚠ Con Required approvals en 0, tú mismo puedes aprobar y hacer merge de tus propios PRs. Si lo pones en 1, GitHub no te permite aprobarte a ti mismo y el merge queda bloqueado.*

1. Ve a tu repositorio en github.com → repo/dojo-python-matricula
2. Pestaña **Settings** (arriba, junto a Insights)
3. En el menú lateral izquierdo, busca la sección **Code and automation** → clic en **Rules** → **Rulesets**



4. Clic en el botón **New ruleset** (esquina superior derecha) → selecciona **New branch ruleset**
5. En **Ruleset Name**, escribe: protect-main
6. En **Enforcement status**, deja **Active**
7. Sección **Target branches** → clic en **Add target** → **Include default branch** (esto selecciona main automáticamente, ya que es la rama por defecto)
8. Sección **Branch rules** — marca estos checkboxes:

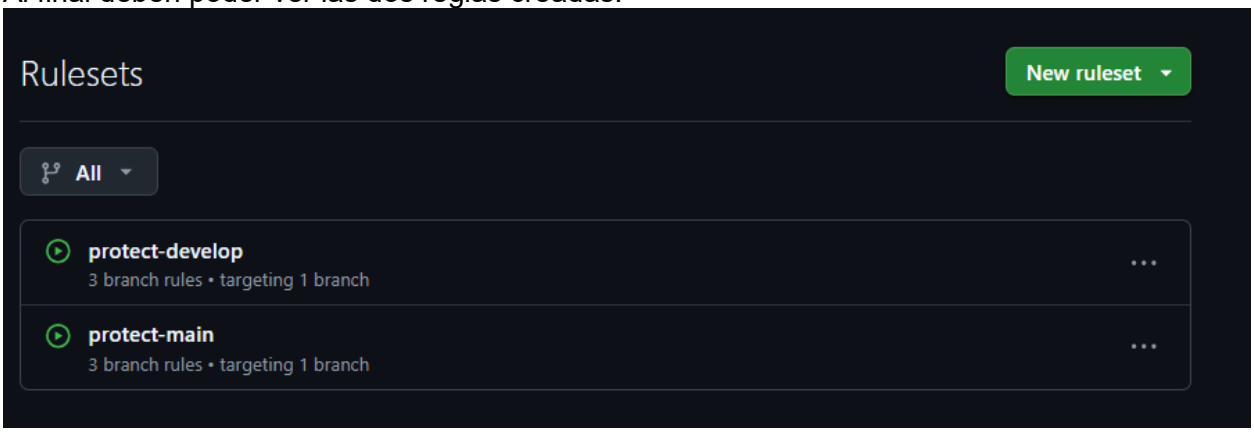
- ☑ **Restrict deletions**
- ☑ **Require a pull request before merging**
  - Al marcarlo aparece un campo **Required approvals** — déjalo en **0**
- ☑ **Block force pushes**

9. Baja hasta el final y clic en **Create**
10. Vuelve a **New ruleset** → **New branch ruleset**
11. Ruleset Name: escribe protect-develop
12. Enforcement status: Active
13. Sección **Target branches** → clic en **Add target** → **Include by pattern** → escribe develop → confirma
14. Marca los mismos tres checkboxes que en el paso 8:

- ☑ Restrict deletions
- ☑ Require a pull request before merging (approvals: 0)
- ☑ Block force pushes

15. Clic en **Create**

Al final deben poder ver las dos reglas creadas:



10.4 Ciclo de desarrollo paso a paso

- Trabajas en tu rama mpastrana haciendo commits
- Creas un PR de mpastrana → develop en GitHub
- El pipeline CI corre automáticamente: ruff + mypy + pytest + SonarCloud
- Si el CI pasa, haces Merge pull request hacia develop
- Creas un PR de develop → main
- Haces Merge pull request hacia main
- El pipeline CD corre automáticamente: construye imágenes Docker, las publica en Docker Hub y despliega en el EC2

Paso 11 — Correcciones y notas importantes

Esta sección recoge los errores más frecuentes que puedes encontrar siguiendo esta guía, con su solución exacta.

11.1 FastAPI genera Swagger automáticamente — Django no

Esta es la diferencia más importante entre los dos frameworks:

Framework	Swagger / OpenAPI	Lo que debes hacer
FastAPI	Generado automáticamente en /docs	Nada. Solo instalar FastAPI y el Swagger aparece solo.
Django REST Framework	No incluido — necesitas drf-spectacular	Instalar drf-spectacular, agregar a INSTALLED_APPS, configurar urls.py con SpectacularAPIView y SpectacularSwaggerView

11.2 Cobertura 0% en Windows con Git Bash

Si pytest --cov reporta que la cobertura es 0% aunque los tests pasen, el problema es que Python no encuentra los módulos en src/. Solución:

```
# Exportar PYTHONPATH antes de ejecutar pytest
export PYTHONPATH=src
pytest --cov=src --cov-report=term-missing

# Alternativa: instalar el paquete en modo editable (ya está en el pyproject.toml)
pip install -e '.[dev]'
```

11.3 mypy muestra errores falsos después de cambios

Si mypy reporta errores que no tienen sentido, limpia la caché y vuelve a ejecutar:

```
rm -rf .mypy_cache
mypy src
```

11.4 SSH RSA rechazado por Amazon Linux 2023

Síntoma: el pipeline CD falla con 'ssh: handshake failed: ssh: unable to authenticate'. Esto ocurre porque Amazon Linux 2023 deshabilita ssh-rsa por política de seguridad. Solución en PuTTY:

```
sudo update-crypto-policies --set LEGACY
sudo systemctl restart sshd

# Verificar que ssh-rsa aparece en la lista (debe estar al final)
sudo sshd -T | grep pubkeyacceptedalgorithms
```

11.5 pscp en Windows con PuTTY

En Windows, scp no funciona directamente con archivos .ppk de PuTTY. Usa pscp que viene instalado con PuTTY:

```
# Formato genérico
pscp -i /ruta/a/tu/clave.ppk /ruta/local/archivo ec2-user@TU_IP:/home/ec2-user/app/

# IMPORTANTE: pscp no interpreta ~/  — usa siempre la ruta absoluta en el destino
# CORRECTO:   ec2-user@IP:/home/ec2-user/app/archivo.yml
# INCORRECTO: ec2-user@IP:~/app/archivo.yml
```

11.6 SonarCloud falla con 'Automatic Analysis is enabled'

Si el job sonar del CI falla con el mensaje 'You are running CI analysis while Automatic Analysis is enabled', entra a SonarCloud y desactiva el Automatic Analysis:

- Ve a sonarcloud.io → tu proyecto → Administration → Analysis Method
- Desactiva el toggle Automatic Analysis
- Vuelve a ejecutar el job en GitHub Actions

11.7 Django: migraciones pendientes en producción

Si el matricula-service no inicia en el EC2 con un error de migración, ejecuta las migraciones manualmente:

```
# Entrar al contenedor del matricula-service
docker exec -it matricula_service_prod bash

# Ejecutar las migraciones pendientes
python src/manage.py migrate

# Salir del contenedor
exit
```

Paso 12 — Verificación final del sistema

12.1 Checklist de entrega

Antes de entregar el dojo, verifica que cada punto funciona correctamente:

Verificación	Comando o URL	Resultado esperado
profesor-service responde	curl http://localhost:8001/profesor-service/health	{"status": "UP"}
matricula-service responde	curl http://localhost:8002/matricula-service/health/	{"status": "UP"}
Swagger FastAPI accesible	http://localhost:8001/profesor-service/docs	Interfaz Swagger con endpoints de profesor
Swagger Django accesible	http://localhost:8002/matricula-service/docs/	Interfaz Swagger con endpoints de estudiante y curso
Tests unitarios pasan	pytest tests/unit -v	All tests passed
Tests integración pasan	pytest tests/integration -v	All tests passed (requiere Docker)
Cobertura >= 80%	pytest --cov=src --cov-fail-under=80	No falla el comando
ruff sin errores	ruff check src tests	All checks passed.
mypy sin errores	mypy src	Success: no issues found
CI verde en GitHub	Actions → CI Pipeline	Todos los jobs en verde
CD desplegó en EC2	Actions → CD Pipeline	Todos los jobs en verde
SonarCloud analiza	sonarcloud.io → tu proyecto	Quality Gate: Passed

12.2 Endpoints disponibles en el sistema completo

Método	Endpoint	Servicio	Descripción
GET	/api/v1/profesores	profesor-service	Lista todos los profesores
POST	/api/v1/profesores	profesor-service	Crea un profesor
GET	/api/v1/profesores/{id}	profesor-service	Obtiene un profesor por id
PUT	/api/v1/profesores/{id}	profesor-service	Actualiza un profesor
DELETE	/api/v1/profesores/{id}	profesor-service	Elimina un profesor
GET	/api/v1/estudiantes	matricula-service	Lista todos los estudiantes
POST	/api/v1/estudiantes	matricula-service	Crea un estudiante
GET	/api/v1/estudiantes/{id}	matricula-service	Obtiene un estudiante por id

Método	Endpoint	Servicio	Descripción
PUT	/api/v1/estudiantes/{id}	matricula-service	Actualiza un estudiante
DELETE	/api/v1/estudiantes/{id}	matricula-service	Elimina un estudiante
GET	/api/v1/cursos	matricula-service	Lista todos los cursos
POST	/api/v1/cursos	matricula-service	Crea un curso